F25-038-D-QCanvas Mid Report

Project Team

Umer Farooq	22I-0891
Hussan Waseem Syed	22I-0893
Muhammad Irtaza Khan	22I-0911

Session 2022-2026

Supervised by

Dr. Imran Ashraf

Co-Supervised by

Dr. Muhammad Nouman Noor

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

April, 2026

Contents

1 Introduction **1**

1.1 Problem Statement 2

1.2 Project Objectives 2

1.3 Project Scope 3

1.3.1 In Scope 3

1.3.2 Out of Scope 3

1.4 Overview of System Modules 4

1.5 User Classes and Characteristics 4

1.6 Assumptions, Dependencies, and Constraints 5

1.6.1 Assumptions 5

1.6.2 Dependencies 5

1.6.3 Constraints 5

2 Project Requirements **6**

2.1 Functional Requirements 6

2.1.1 Conversion Module 6

2.1.2 Simulation Module 7

2.1.3 Hybrid CPU–QPU Module 7

2.1.4 Web IDE Module 8

2.1.5 API and Realtime 8

2.1.6 Validation 9

2.2 Non-Functional Requirements 9

2.2.1 Reliability 9

2.2.2 Usability 9

2.2.3 Performance 10

2.2.4 Security 10

2.3 External Interface Requirements 11

2.3.1 User Interfaces 11

2.3.1.1 Web IDE Interface 11

2.3.1.2 Results Visualization 11

2.3.2 Hardware Interfaces 11

2.3.3 Software Interfaces 11

2.3.3.1 Quantum Framework APIs 11
2.3.3.2 Database Interface 12
2.3.3.3 External Services 12
2.3.4 Communication Interfaces 12
2.3.4.1 REST API 12
2.3.4.2 WebSocket Interface 12
2.4 Use-Cases 13
2.4.1 Functional Requirements Traceability 13
2.4.2 UC-01: Launch Web IDE 14
2.4.3 UC-02: Write Circuit Code 14
2.4.4 UC-03: Convert to OpenQASM 14
2.4.5 UC-04: Execute Circuit 15
2.4.6 UC-05: Visualize Results 15
2.4.7 UC-06: Handle Validation Errors 16
2.4.8 UC-07: Export OpenQASM 16
2.4.9 UC-08: Load Example Circuit 17
2.4.10 UC-09: Query Backend Capabilities 17
2.4.11 UC-10: Hybrid CPU–QPU Execution 18
2.5 Use Cases Diagram 19
2.6 Domain Model Diagram 20

3 System Overview **21**
3.1 Architectural Design 21
3.1.1 Layered Responsibilities 21
3.1.2 Technology Stack 22
3.1.2.1 Frontend Technologies 22
3.1.2.2 Backend Technologies 22
3.1.2.3 Quantum Computing Stack 22
3.1.3 Hybrid CPU–QPU Execution Model 23
3.1.3.1 Orchestration Boundary 23
3.1.3.2 Data Flow 23
3.1.3.3 Execution Scope 23
3.2 Design Models 24
3.2.1 Activity Diagrams 24
3.2.1.1 Activity Diagram: Circuit Conversion and Execution . . 24
3.2.1.2 Activity Diagram: User Registration 25
3.2.2 Sequence Diagrams 25
3.2.2.1 Sequence Diagram: Unified Execution Flow 26
3.2.2.2 Sequence Diagram: Authentication 26
3.2.2.3 System Sequence Diagrams (SSD) 27

3.2.3 Architecture Diagram 31
3.2.4 Component Diagram 32
3.2.5 Package Diagram 33
3.2.6 Deployment Diagram 34
3.2.7 State Diagrams 35
3.2.7.1 State Diagram: Conversion Process 35
3.2.7.2 State Diagram: Simulation Job 36
3.2.8 Data Flow Diagrams (DFD) 37
3.2.8.1 DFD Level 0 (Context Diagram) 37
3.2.8.2 DFD Level 1 38
3.3 Data Design 38
3.3.1 Information Domain Transformation 38
3.3.1.1 Core Entities 38
3.3.1.2 Entity Relationship Diagram (ERD) 39
3.3.2 Data Organization and Processing 39
3.3.3 Data Storage Mechanisms 40
3.3.3.1 Current Implementation 40
3.3.4 API Data Models 40
3.3.4.1 Request Models 41
3.3.4.2 Response Models 41

4 Implementation and Results **42**
4.1 Sprint 3 Deliverables 42
4.1.1 Interactive Circuit Visualization 42
4.1.2 Quantum Explain It Mode 43
4.1.3 Gamified Learning and Community Features 43
4.1.4 Extensibility and New Framework Onboarding 43
4.2 Implementation Progress 44
4.2.1 Iteration 1: Core Infrastructure and Basic Features 44
4.2.1.1 Core Infrastructure 44
4.2.1.2 Quantum Features 44
4.2.2 Iteration 2: Advanced Features and Hybrid Execution 44
4.2.2.1 Advanced Language Features 45
4.2.2.2 PennyLane Integration Expansion 45
4.2.2.3 User Management and Security (CIA Compliance) . . . 45
4.2.2.4 Persistence and Database Layer 45
4.2.2.5 API for QSim Compilation (Hybrid CPU/QPU) 46
4.3 User Interface Implementation 46
4.4 Testing and Validation 47
4.4.1 Testing Strategy 47

4.4.2 Sprint 3 Verification 47
4.4.3 Test Coverage 48
4.4.4 Validation Results 48
4.5 Evaluation and Benchmarking 48
4.5.1 Performance Evaluation 48
4.5.2 Correctness Benchmarking 49

5 Conclusion **50**
5.1 Future Work 50
5.1.1 Iteration 4: Refinement and Stabilization (Months 5-6) 50
5.1.1.1 Educational Intelligence Features 51
5.1.1.2 Community and Sharing Platform 51
5.1.1.3 Platform Extensibility 52
5.1.2 Iteration 5: Production Readiness and Advanced Analytics (Months 7-8) 53
5.1.2.1 Deep QSim Integration 53
5.1.2.2 Public API and Developer Tools 54
5.1.2.3 Advanced Visualization and Analytics 54
5.1.2.4 Production Infrastructure 55
5.1.3 Long-Term Research Directions 56
5.1.3.1 Quantum Algorithm Optimization 57
5.1.3.2 Advanced Educational Features 57
5.1.3.3 Research Collaboration Tools 57
5.1.4 Conclusion 58

List of Figures

2.1 Use Case Diagram 19
2.2 Domain Model 20

3.1 Activity Diagram — Circuit Conversion and Execution 24
3.2 Activity Diagram — User Registration 25
3.3 Sequence Diagram — Unified Flow 26
3.4 Sequence Diagram — Authentication 27
3.5 SSD — Authentication 28
3.6 SSD — Conversion 29
3.7 SSD — Simulation 30
3.8 Architecture Diagram 31
3.9 Component Diagram 32
3.10 Package Diagram 33
3.11 Deployment Diagram 34
3.12 State Diagram — Conversion 35
3.13 State Diagram — Job 36
3.14 DFD Level 0 37
3.15 DFD Level 1 38
3.16 Entity Relationship Diagram 39

List of Tables

1.1 User Classes and Characteristics 4

2.1 Traceability between FRs and UCs 13

3.1 Data Organization and Processing 40

Chapter 1

Introduction

The field of quantum computing has transitioned significantly from theoretical abstraction to the era of Noisy Intermediate-Scale Quantum (NISQ) devices [1, 2]. As hardware vendors like IBM, Google, and Rigetti race to increase qubit counts and fidelity, the accompanying software ecosystem has expanded rapidly. However, this growth has come at the cost of fragmentation. Researchers and developers today are confronted with a "Tower of Babel" scenario, where competing frameworks—such as Qiskit, Cirq, and PennyLane—operate in silos, each with unique syntax, compilation targets, and visualization tools [3, 4, 5].

This fragmentation poses a barrier to entry for students and creates friction for experienced researchers who wish to compare algorithms across different hardware baselines. The lack of a unified development environment that can transparently bridge these frameworks means that users often spend more time wrestling with proprietary APIs than designing quantum logic. Furthermore, the absence of standardized intermediate representations in educational tools obscures the critical translation step where high-level Python code becomes executable quantum instructions.

"QCanvas" is proposed as a definitive solution to this interoperability challenge. It is designed as a unified, standard-compliant simulation platform that serves as a universal translator and execution engine for quantum circuits. By centering its architecture on OpenQASM 3.0 [6, 7], QCanvas provides a transparent window into quantum execution, allowing code from any major framework to be parsed, visualized, and simulated in a consistent environment.

This document details the complete software requirements specification for QCanvas. It outlines the architectural decisions and functional capabilities intended to support developers, researchers, and educators in overcoming the current limitations of the quantum software landscape.

1.1 Problem Statement

Despite rapid progress in NISQ-era hardware and software, there is no unified simulation platform that allows quantum circuits from major frameworks (Qiskit, Cirq, PennyLane) to be expressed, compared, and executed through a single, standards-based workflow. Learners and researchers are forced to navigate duplicated concepts, incompatible APIs, and fragmented tooling across frameworks and simulators.

Specifically, the current ecosystem lacks:

- **A unified intermediate representation in tooling:** Most frameworks hide or only partially expose the compiled quantum instructions, making it difficult to reason about what actually runs on the backend.
- **A single, framework-agnostic IDE:** There is no web-based environment where users can work with Cirq, Qiskit, and PennyLane side by side while seeing the same OpenQASM 3.0 view of their circuits.
- **A consistent simulation interface:** Users must switch between different simulators, configuration styles, and result formats when moving across frameworks.
- **Beginner-friendly feedback:** Existing professional tools often prioritize flexibility over clarity, leaving students without guided error reporting or visual explanations.

QCanvas addresses this problem by providing a unified interface where users can write in their preferred framework but understand and execute their logic through a common, standardized lens (OpenQASM 3.0) [6]. The remainder of this document formalizes the scope, objectives, requirements, and architecture of that solution.

1.2 Project Objectives

The primary goal of QCanvas is to democratize access to quantum programming through a simulation-first approach. The specific objectives are:

- **To Unify Quantum Workflows:** Create a single platform capable of parsing and converting code from major frameworks (Cirq, Qiskit, PennyLane) into a universal format (OpenQASM 3.0).
- **To Enhance Educational Clarity:** Provide a web-based IDE that offers immediate feedback, converting abstract Python code into clear quantum assembly and visual circuit diagrams.

- **To Ensure Standard Compliance:** Validate all generated code against a strict OpenQASM 3.0 subset to ensure correctness and portability.
- **To Enable Hybrid Simulation:** Implement a robust architecture where classical orchestration (CPU) and quantum execution (Simulators) are clearly separated, mimicking real-world hybrid quantum-classical systems.

1.3 Project Scope

The scope of QCanvas is defined to deliver a functional, robust simulation environment within the project timeline.

1.3.1 In Scope

The system shall:

- **Conversion Engine:** Robustly parse Python-based quantum circuits and convert them to OpenQASM 3.0 AST and source code.
- **Validation Layer:** Semantic analysis to ensure generated code adheres to the supported gate set and control flow limitations.
- **Simulation Backends:** Support for Statevector, Density Matrix, and Stabilizer simulations with configurable shot counts.
- **Interactive IDE:** A Monaco-based web editor for writing code, visualizing results (histograms), and managing projects.
- **Real-Time Feedback:** WebSocket-based streaming for long-running batch conversions or heavy simulations.

1.3.2 Out of Scope

To ensure feasibility, the following are explicitly excluded:

- **Hardware Execution:** Direct integration with physical QPUs (e.g., IBM Quantum Experience) is not part of this release.
- **Pulse-Level Control:** Low-level control of microwave pulses (OpenPulse) is outside the scope.

- **QEC Protocols:** Specialized quantum error correction encoding/decoding layers are not implemented.
- **Physical Qubit Mapping:** Manual addressing of physical qubit layouts (e.g., `q[0] -> $0`) is not supported.

1.4 Overview of System Modules

The system is architected into modular components to ensure scalability and maintainability:

- **Conversion Module:** The core engine responsible for AST parsing, framework interoperability, and code generation.
- **Simulation Module:** The execution engine handling the mathematics of state evolution and measurement.
- **Validation Module:** A gatekeeper ensuring code correctness before execution.
- **Web IDE Module:** The frontend interface handling user interaction, visualization, and state management.
- **API Module:** The communication layer exposing REST and WebSocket endpoints.

1.5 User Classes and Characteristics

QCanvas serves a diverse audience with varying needs:

User Class	Primary Goals	Key Needs
Beginners	Build visual intuition and learn quantum basics.	Templates, guided error messages, simple visual feedback.
Researchers	Reproduce experiments and compare frameworks.	Batch processing, exact statevector access, cross-framework analysis.
Educators	Demonstrate concepts effectively to students.	Shareable examples, clean visualization, standard syntax compliance.

Table 1.1: User Classes and Characteristics

1.6 Assumptions, Dependencies, and Constraints

1.6.1 Assumptions

It is assumed that users have basic familiarity with Python syntax. The system assumes a stable modern browser environment (Chrome/Firefox/Edge) for the IDE.

1.6.2 Dependencies

Key external dependencies include the `Python 3.11+` runtime, `FastAPI` for the backend, `Next.js` for the frontend, and the quantum libraries `Cirq`, `Qiskit`, and `PennyLane`.

1.6.3 Constraints

System performance is constrained by host memory for Statevector simulations (scaling exponentially with qubit count). API response times are targeted at under 2 seconds for standard conversion tasks.

Chapter 2

Project Requirements

This chapter describes the comprehensive functional and non-functional requirements for QCanvas, followed by use-cases, external interface specifications, and system features that define the complete system behavior.

2.1 Functional Requirements

The functional requirements below are grouped by subsystem so that the conversion, simulation, hybrid orchestration, and UI responsibilities remain traceable to individual design decisions. Each group reflects the behavior that is already implemented or directly supported by the Sprint 3 architecture.

2.1.1 Conversion Module

The conversion module requirements define how source circuits are parsed, normalized, validated, and translated into OpenQASM 3.0. They also capture the statistics and diagnostics required for educational feedback and regression analysis.

- FR-C1: The system shall parse Cirq, Qiskit, and PennyLane circuits using AST-based parsing with runtime fallback.
- FR-C2: The system shall generate OpenQASM 3.0 code conforming to the supported subset per project scope [6].
- FR-C3: The system shall handle measurement and reset instructions correctly.
- FR-C4: The system shall expose conversion statistics including qubits, depth, and gate counts.

- FR-C5: The system shall validate generated OpenQASM 3.0 against the supported subset and provide detailed diagnostics [7].
- FR-C6: The system shall support gate mapping with parameter and broadcast handling.

2.1.2 Simulation Module

The simulation module requirements capture the execution behavior of the backends that evaluate OpenQASM output. These requirements emphasize determinism, backend selection, and the metadata needed to interpret results.

- FR-S1: The system shall support statevector, density matrix, and stabilizer simulation backends [2].
- FR-S2: The system shall allow configurable shots (1–10,000) and return counts and metadata.
- FR-S3: The system shall cache identical executions to improve performance.
- FR-S4: The system shall allow seed configuration for deterministic runs.
- FR-S5: The system shall provide backend capability discovery (max qubits, supported gates).
- FR-S6: The system shall return results with counts, execution time, and optional state snapshots.

2.1.3 Hybrid CPU–QPU Module

The hybrid module requirements describe how QCanvas separates classical orchestration from quantum execution. This boundary is important for keeping the frontend responsive while heavier work is delegated to the backend runtime.

- FR-H1: The system shall separate classical orchestration (CPU) from quantum circuit execution (QPU/Simulator) [8].
- FR-H2: The system shall implement data flow: Source $\rightarrow$ QASM $\rightarrow$ Execution $\rightarrow$ Results.
- FR-H3: The system shall provide WebSocket progress updates for long-running operations.

- FR-H4: The system shall enforce simulation-first execution with optional pluggable endpoints.
- FR-H5: The system shall provide resource-aware execution with limits and timeouts.

2.1.4 Web IDE Module

The web IDE requirements focus on the user-facing editor, visualization, and project-management features that make the platform usable as an interactive learning environment rather than only a conversion utility.

- FR-W1: The system shall provide code editing with syntax highlighting and diagnostics.
- FR-W2: The system shall display circuit visualization and results panels.
- FR-W3: The system shall support multiple example projects and templates.
- FR-W4: The system shall provide keyboard shortcuts for frequent actions.
- FR-W5: The system shall support multi-file projects with workspace management.

2.1.5 API and Realtime

The API and realtime requirements ensure that the platform can support both interactive use in the browser and programmatic use through external clients. They also provide the event stream needed for long-running operations.

- FR-A1: The system shall expose REST endpoints for conversion and validation.
- FR-A2: The system shall expose WebSocket endpoints for progress streaming.
- FR-A3: The system shall provide structured error payloads with codes and hints.
- FR-A4: The system shall provide discovery endpoints for frameworks and backends.
- FR-A5: The system shall support batch conversion and simulation operations.

2.1.6 Validation

Validation requirements are kept separate because they guard every other module. They ensure that unsupported language constructs are identified early and that users receive actionable feedback instead of silent failures.

- FR-V1: The system shall validate OpenQASM 3.0 output against the allowed subset.
- FR-V2: The system shall suggest nearest supported constructs for incompatibilities.
- FR-V3: The system shall emit precise diagnostics with line/column, code, and remediation.
- FR-V4: The system shall check for excluded features (timing, pulse-level) with clear guidance.

2.2 Non-Functional Requirements

The non-functional requirements focus on reliability, usability, performance, and security. These constraints shape the implementation choices for the editor, backend services, and execution pipeline.

2.2.1 Reliability

- **NFR-R1: Service Availability**
  The system containers (Backend, Database, Redis) shall report a 'healthy' status code (200 OK) on their health check endpoints 99.9% of the time during active operation.
- **NFR-R2: Audit Trail**
  The system shall record an audit log entry in the `ApiActivity` table for 100% of API requests resulting in 4xx or 5xx status codes.

2.2.2 Usability

- **NFR-U1: Keyboard Efficiency**
  The system shall execute the current quantum circuit when the user triggers the `Ctrl+Shift+R` (or `Cmd+Shift+R`) key combination.

- **NFR-U2: Visual Accessibility**
  The user interface shall utilize the Monaco Editor's high-contrast `vs-dark` or `vs` theme standards to ensure text legibility.

2.2.3 Performance

- **NFR-P1: Conversion Latency**
  The API shall process circuit conversion requests (AST parsing to QASM generation) in under 1500ms for circuits with <100 gates.
- **NFR-P2: Simulation Throughput**
  The simulator shall complete execution of standard algorithms (e.g., Grover's Search n=2) in under 5 seconds.

2.2.4 Security

- **NFR-S1: API Rate Limiting**
  The API shall strictly enforce a rate limit of **20 requests per minute** per client IP address for conversion endpoints.
- **NFR-S2: Data Integrity Protection**
  The system shall sanitize 100% of user inputs via ORM parameterization to prevent SQL injection vulnerabilities.

2.3 External Interface Requirements

2.3.1 User Interfaces

2.3.1.1 Web IDE Interface

- Monaco-based code editor with syntax highlighting for Qiskit, Cirq, PennyLane, and OpenQASM 3.0
- Multi-panel layout: Editor, Results, Console, Logs, Statistics
- File tree navigation with create, rename, delete operations
- Theme switcher (dark/light modes)
- Responsive design for desktop and tablet devices

2.3.1.2 Results Visualization

- Tabbed results display: Console, Output, Histogram, OpenQASM, Errors, Stats
- Interactive measurement histograms using Chart.js
- Real-time execution status updates
- Export functionality for results (JSON, CSV)

2.3.2 Hardware Interfaces

The system does not interface with physical quantum hardware. All execution is simulation-based through software backends.

2.3.3 Software Interfaces

2.3.3.1 Quantum Framework APIs

- Qiskit API (version 0.44+): Circuit construction and manipulation [3]
- Cirq API (version 1.0+): Circuit operations and gate definitions [4]
- PennyLane API (version 0.33+): Quantum node processing [5]

2.3.3.2 Database Interface

- PostgreSQL (future): User data, projects, circuit storage
- Redis (future): Session caching and result caching

2.3.3.3 External Services

- No external quantum hardware services in current scope
- Future: Pluggable QPU endpoints with same API contract

2.3.4 Communication Interfaces

2.3.4.1 REST API

- Protocol: HTTPS in production, HTTP in development
- Format: JSON request/response bodies
- Authentication: JWT tokens (future implementation)
- Base URL: /api/v1/

2.3.4.2 WebSocket Interface

- Protocol: WSS in production, WS in development
- Format: JSON message payloads
- Endpoint: /ws
- Use cases: Progress updates, real-time notifications

2.4 Use-Cases

The following use cases describe the core interactions between users and the QCanvas system.

2.4.1 Functional Requirements Traceability

Table 2.1 summarizes how the main functional requirement groups (Conversion, Simulation, Hybrid, Web IDE, API/Realtime, Validation) are exercised by the detailed use cases (UC-01–UC-10). This ensures that each requirement is covered by at least one concrete scenario and that use-case identifiers remain consistent with the diagrams.

FR Group	Primary Related Use-Cases
Conversion (FR-C1–FR-C6)	UC-02: Write Circuit Code UC-03: Convert to OpenQASM UC-06: Handle Validation Errors UC-07: Export OpenQASM
Simulation (FR-S1–FR-S6)	UC-04: Execute Circuit UC-05: Visualize Results UC-09: Query Backend Capabilities UC-10: Hybrid CPU–QPU Execution
Hybrid CPU–QPU (FR-H1–FR-H5)	UC-04: Execute Circuit UC-10: Hybrid CPU–QPU Execution
Web IDE (FR-W1–FR-W5)	UC-01: Launch Web IDE UC-02: Write Circuit Code UC-05: Visualize Results UC-08: Load Example Circuit
API and Realtime (FR-A1–FR-A5)	UC-03: Convert to OpenQASM UC-04: Execute Circuit UC-05: Visualize Results UC-09: Query Backend Capabilities
Validation (FR-V1–FR-V4)	UC-03: Convert to OpenQASM (error paths) UC-04: Execute Circuit (error paths) UC-06: Handle Validation Errors

Table 2.1: Traceability between FRs and UCs

2.4.2 UC-01: Launch Web IDE

- **Primary Actor:** User
- **Precondition:** Browser has network access to the QCanvas URL.
- **Main Flow:**
  1. User navigates to the application URL.
  2. System loads the Monaco editor, sidebar, and results panel.
  3. System fetches the default "Hello World" circuit.
  4. System displays the "Ready" status indicator.
- **Alternate Flows:**

  2a *Server Unreachable:* System displays a 404 error message with retry option.
- **Postcondition:** IDE is initialized and waiting for input.

2.4.3 UC-02: Write Circuit Code

- **Primary Actor:** User
- **Precondition:** UC-01 complete; Editor is active.
- **Main Flow:**
  1. User selects a target framework (e.g., Qiskit) from the dropdown.
  2. User types quantum circuit code into the editor.
  3. System performs real-time syntax checking (linting).
- **Alternate Flows:**

  2a *Syntax Error:* System highlights invalid code.
- **Postcondition:** Valid or invalid source code exists in the editor buffer.

2.4.4 UC-03: Convert to OpenQASM

- **Primary Actor:** User
- **Precondition:** Source code is present in the editor.
- **Main Flow:**

1. User clicks the "Compile" button.
2. System sends source code to the Conversion API.
3. API parses the code and generates OpenQASM 3.0 intermediate representation.
4. System returns the QASM code and displays it in the output panel.

- **Alternate Flows:**

  3a *Syntax Error:* Source code is invalid. API returns line number. System highlights the error line in red.

  3b *Unsupported Gate:* Source uses a gate not in the supported subset. System returns gate not supported error.

- **Postcondition:** OpenQASM 3.0 code is available for simulation.

2.4.5 UC-04: Execute Circuit

- **Primary Actor:** User
- **Precondition:** Valid OpenQASM 3.0 code exists (from UC-03 or manual entry).
- **Main Flow:**

  1. User clicks "Run" button.
  2. User configures parameters (Shots=1024, Backend=Statevector).
  3. System submits job to Queue.
  4. Simulation Backend executes the circuit.
  5. System pushes results to the frontend via WebSocket.

- **Alternate Flows:**

  4a *Timeout:* Simulation takes > 30s. System displays timeout message.

- **Postcondition:** Measurement counts and state data are available in client state.

2.4.6 UC-05: Visualize Results

- **Primary Actor:** User
- **Precondition:** UC-04 completed successfully.
- **Main Flow:**

1. System receives JSON result payload.
2. System renders a bar chart using Chart.js for measurement counts.
3. System displays execution metadata (time, backend version).

- **Alternate Flows:**

  2a *Empty Result:* If 0 shots were requested, System displays statevector only without histogram.

- **Postcondition:** Results are visible to the user.

2.4.7 UC-06: Handle Validation Errors

- **Primary Actor:** User
- **Precondition:** User has triggered Conversion or Execution with invalid input.
- **Main Flow:**

  1. System detects semantic error (e.g., unsupported gate).
  2. System squiggles the error line in red in the editor.
  3. System displays a "Problems" pane with error codes (e.g., ERR-020).

- **Alternate Flows:**

  3a *Manual Correction:* User edits the code to resolve the issue. System re-validates and clears the error indicator.

- **Postcondition:** User is aware of the exact location and nature of the error.

2.4.8 UC-07: Export OpenQASM

- **Primary Actor:** User
- **Precondition:** UC-03 completed successfully.
- **Main Flow:**

  1. User clicks the "Download QASM" icon.
  2. System generates a .qasm file including metadata header.
  3. System triggers the browser file download dialog.

- **Alternate Flows:**

3a *File Size Limit:* If code > 1MB, System zips the content before download.

- **Postcondition:** The OpenQASM source file is saved to the User's local file system.

2.4.9 UC-08: Load Example Circuit

- **Primary Actor:** User
- **Precondition:** IDE is active.
- **Main Flow:**
  1. User opens the "Examples" drawer.
  2. User clicks on "Grover's Algorithm".
  3. System replaces current editor content with Grover's source code.
  4. System displays a modal with a brief explanation of the algorithm.
- **Alternate Flows:**

  3a *Dirty Buffer:* If current code is unsaved, System prompts "Discard changes?".
- **Postcondition:** Example code is loaded and ready to run.

2.4.10 UC-09: Query Backend Capabilities

- **Primary Actor:** Developer
- **Precondition:** Authenticated API access.
- **Main Flow:**
  1. Developer requests `GET /api/v1/backends`.
  2. System returns list of available backends (e.g., "statevector").
  3. System details max_qubits (e.g., 20) and supported_gates.
- **Alternate Flows:**

  2a *Database Down:* System returns cached fallback capabilities list.
- **Postcondition:** Developer client is self-configured to respect backend limits.

2.4.11 UC-10: Hybrid CPU–QPU Execution

- **Primary Actor:** User/System
- **Precondition:** Circuit submitted for execution.
- **Main Flow:**
  1. **Orchestration (CPU):** Validates QASM and allocates a Job ID.
  2. **Orchestration (CPU):** Dispatches payload to QSim Service.
  3. **Execution (QSim):** Initializes Quantum Statevector.
  4. **Execution (QSim):** Applies Gate Operations sequentially.
  5. **Orchestration (CPU):** Receives final counts and saves to DB.
- **Alternate Flows:**

  3a *Memory OOM:* QSim detects >20 qubits. Returns "Resource Limit Exceeded". CPU relays error to user.
- **Postcondition:** Job marked as "Completed" or "Failed" in database.

2.5 Use Cases Diagram

This diagram outlines the primary actors and their interactions with QCanvas across core flows like conversion, simulation, and results visualization.

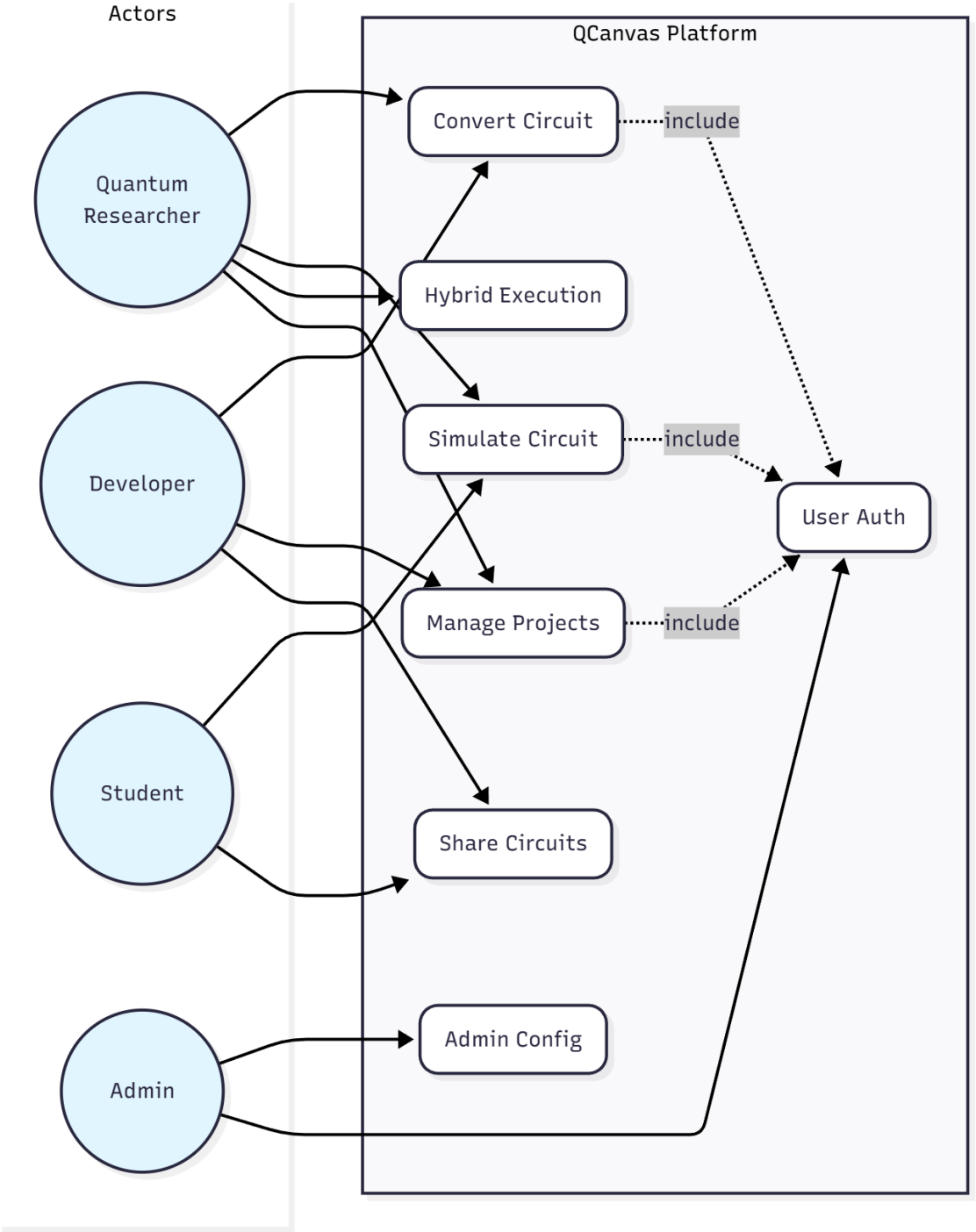

Figure 2.1: Use Case Diagram

2.6 Domain Model Diagram

This diagram shows the core entities (e.g., Circuit, ConversionRequest, SimulationJob, SimulationResult, User, Project) and key relationships that support conversion and simulation.

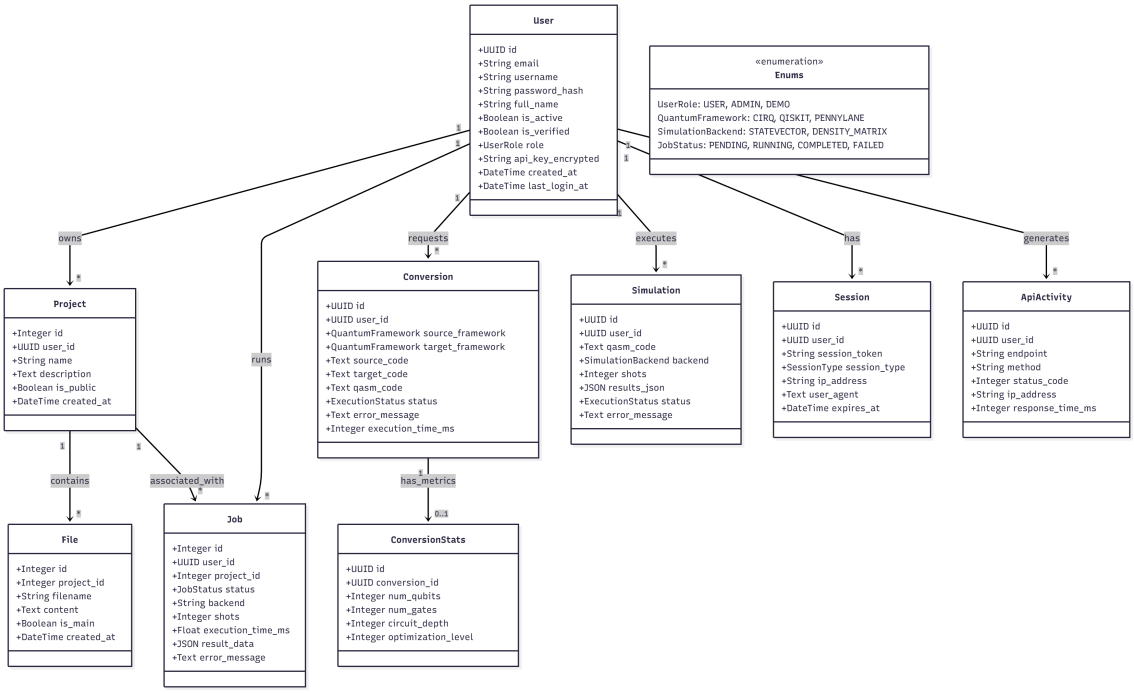

Figure 2.2: Domain Model

Chapter 3

System Overview

The system architecture of QCanvas follows a modular layered design separating conversion, validation, simulation, and UI concerns. This design ensures scalability, maintainability, and flexibility while supporting the hybrid CPU–QPU execution model [8].

3.1 Architectural Design

QCanvas employs a hybrid architecture where the Next.js frontend handles UI and simple operations, while the FastAPI backend manages complex quantum computations. QCanvas orchestrates compilation and validation; QSim executes circuits on simulation backends.

3.1.1 Layered Responsibilities

- **Presentation Layer (Next.js)**: Editor UI, circuit visualization, results dashboards, accessibility features, and client-side state management.
- **API Layer (FastAPI)**: REST endpoints for conversion/validation/simulation, WebSocket for progress streams, structured error handling, and request validation.
- **Conversion Layer (QCanvas)**: AST parsing, intelligent gate mapping, OpenQASM 3.0 generation, validation, analytics, and logging.
- **Execution Layer (QSim)**: Simulation backends (statevector, density matrix, stabilizer), job scheduling, result aggregation, caching, and metrics.
- **Persistence Layer**: PostgreSQL for user data and projects (future), Redis for caching and sessions (future).

- **Observability Layer**: Health checks, Prometheus metrics, execution time histograms, and structured logging.

3.1.2 Technology Stack

3.1.2.1 Frontend Technologies

- Next.js 14 with App Router for server-side rendering and routing
- React 18 with TypeScript for type-safe UI development
- Tailwind CSS for styling with custom quantum-themed design
- Monaco Editor for code editing with syntax highlighting
- Zustand for lightweight state management
- Chart.js for measurement histogram visualization
- WebSocket client for real-time updates

3.1.2.2 Backend Technologies

- Python 3.11+ for backend logic
- FastAPI for high-performance async API framework
- Uvicorn ASGI server for production deployment
- Pydantic for data validation and serialization
- SQLAlchemy ORM with PostgreSQL (future)
- Redis for caching and session management (future)

3.1.2.3 Quantum Computing Stack

- Cirq 1.0+ for Google quantum framework support [4]
- Qiskit 0.44+ for IBM quantum framework support [3]
- PennyLane 0.33+ for Xanadu quantum framework support [5]
- Custom OpenQASM 3.0 generator (QASM3Builder) [6]
- Custom simulation backends (statevector, density matrix, stabilizer) [2]

3.1.3 Hybrid CPU–QPU Execution Model

3.1.3.1 Orchestration Boundary

- **CPU Responsibilities**: Circuit construction, parameter preparation, AST parsing, OpenQASM generation, validation, pre/post-processing, result aggregation.
- **QPU/Simulator Responsibilities**: Gate-level circuit execution, statevector/density matrix evolution, measurement sampling, state snapshot generation.

3.1.3.2 Data Flow

1. User writes circuit in source framework (Qiskit/Cirq/PennyLane)
2. Frontend sends code to backend via REST API
3. Backend parses code using AST or runtime execution
4. QCanvas generates OpenQASM 3.0 intermediate representation
5. Validation checks conformance to supported subset
6. QSim executes circuit on selected backend
7. Results aggregated and returned via API
8. Frontend visualizes results (histograms, statistics)
9. WebSocket provides real-time progress updates

3.1.3.3 Execution Scope

- Simulation-first: All execution through software simulators
- No pulse-level control or timing features
- Future pluggable QPU endpoints behind same API contract
- Intelligent caching for identical circuit executions

3.2 Design Models

The diagrams below are grouped by behavior rather than by file location so the reader can connect each model to the corresponding runtime concern without jumping back and forth across the chapter.

3.2.1 Activity Diagrams

The activity diagrams summarize the end-to-end user journeys that matter most in Sprint 3: circuit conversion, execution, and onboarding. They make the validation and orchestration boundaries visible before the reader reaches the detailed component diagrams.

3.2.1.1 Activity Diagram: Circuit Conversion and Execution

This diagram presents the end-to-end flow from code input in the Next.js editor, through FastAPI conversion and validation services, into QSim execution and results visualization.

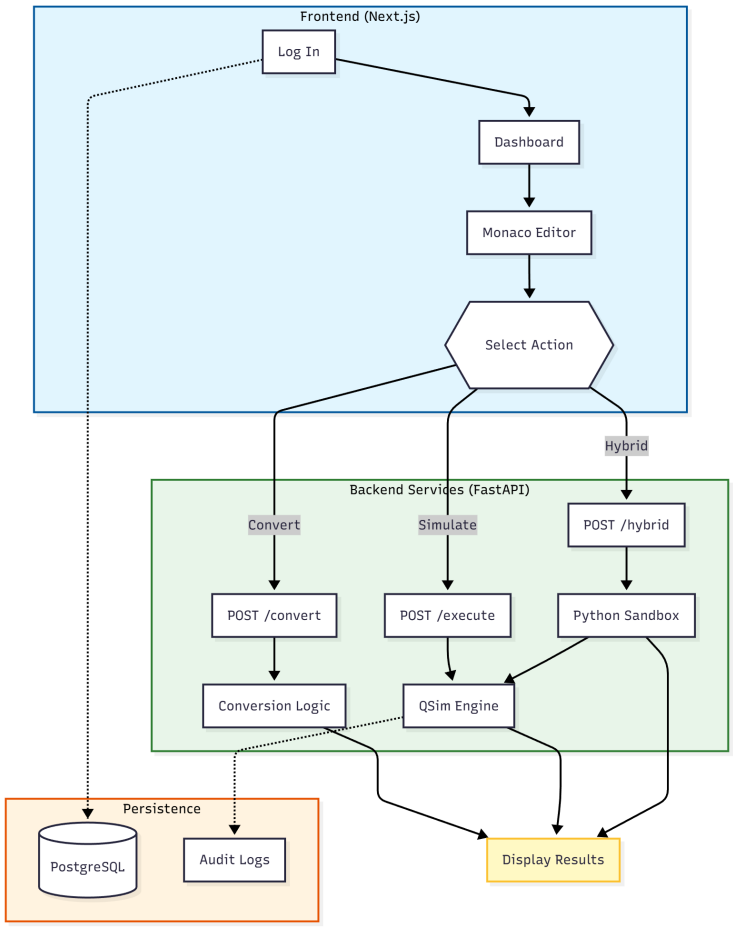

Figure 3.1: Activity Diagram — Circuit Conversion and Execution

3.2.1.2 Activity Diagram: User Registration

This diagram captures the user registration and onboarding flow implemented in the authentication module.

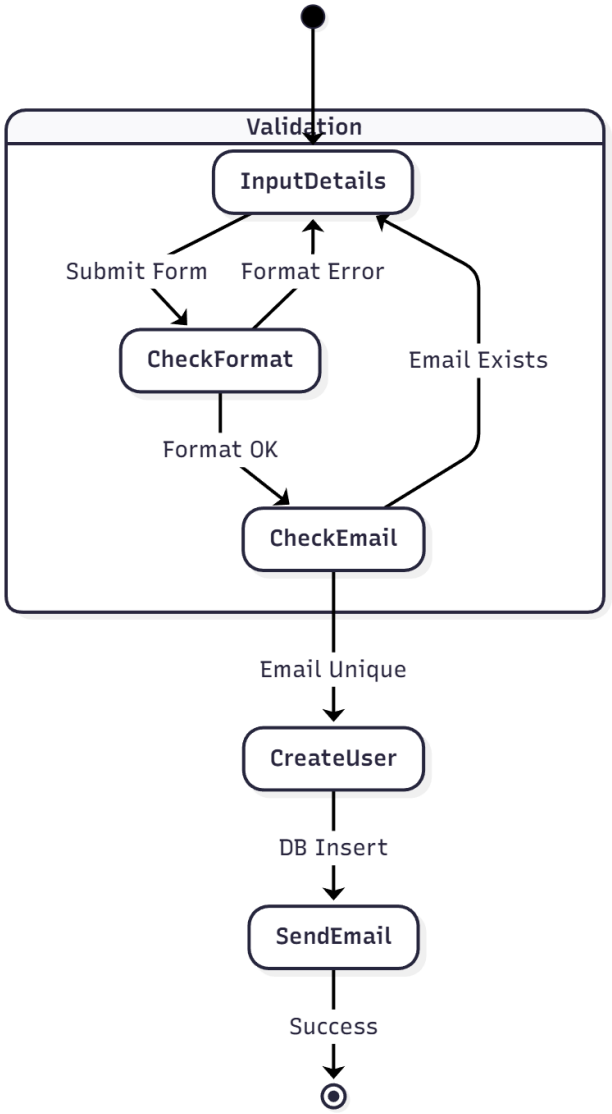

Figure 3.2: Activity Diagram — User Registration

3.2.2 Sequence Diagrams

The sequence diagrams focus on interactions between concrete software components. They are especially useful in QCanvas because the same user action can trigger the editor, converter, validator, simulator, and result renderer in one path.

3.2.2.1 Sequence Diagram: Unified Execution Flow

This diagram shows the unified interaction flow across concrete system components.

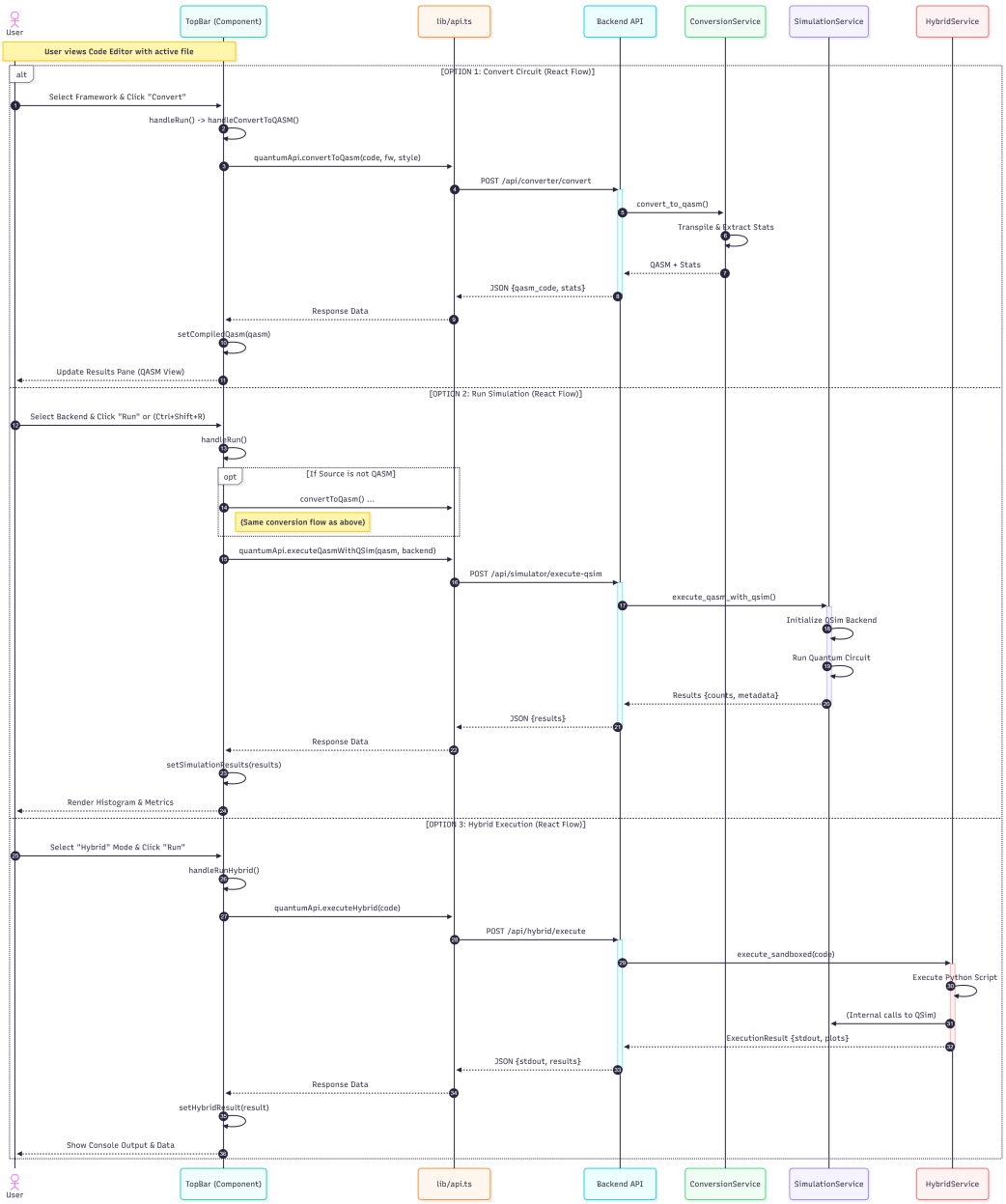

Figure 3.3: Sequence Diagram — Unified Flow

3.2.2.2 Sequence Diagram: Authentication

This diagram details the authentication process involving the frontend, API, and Database.

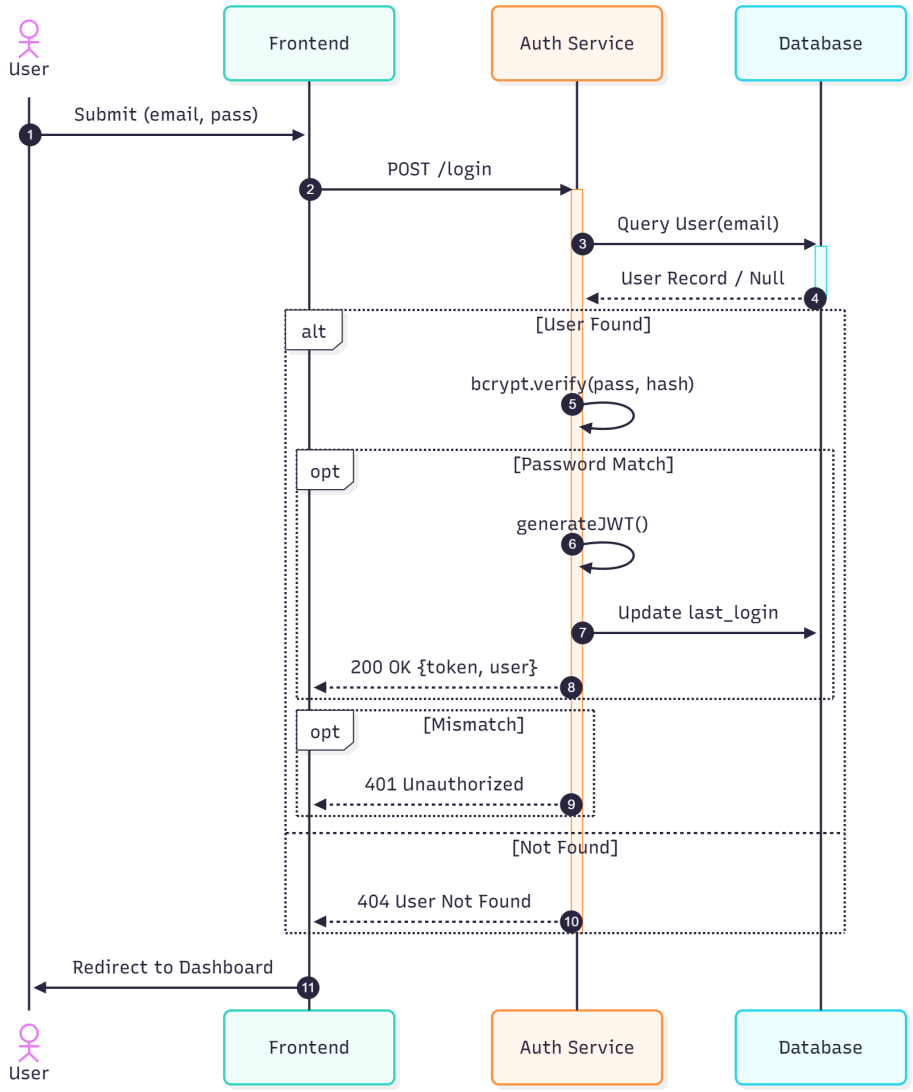

Figure 3.4: Sequence Diagram — Authentication

3.2.2.3 System Sequence Diagrams (SSD)

The following diagrams illustrate the system event flows for key scenarios.

SSD: Authentication

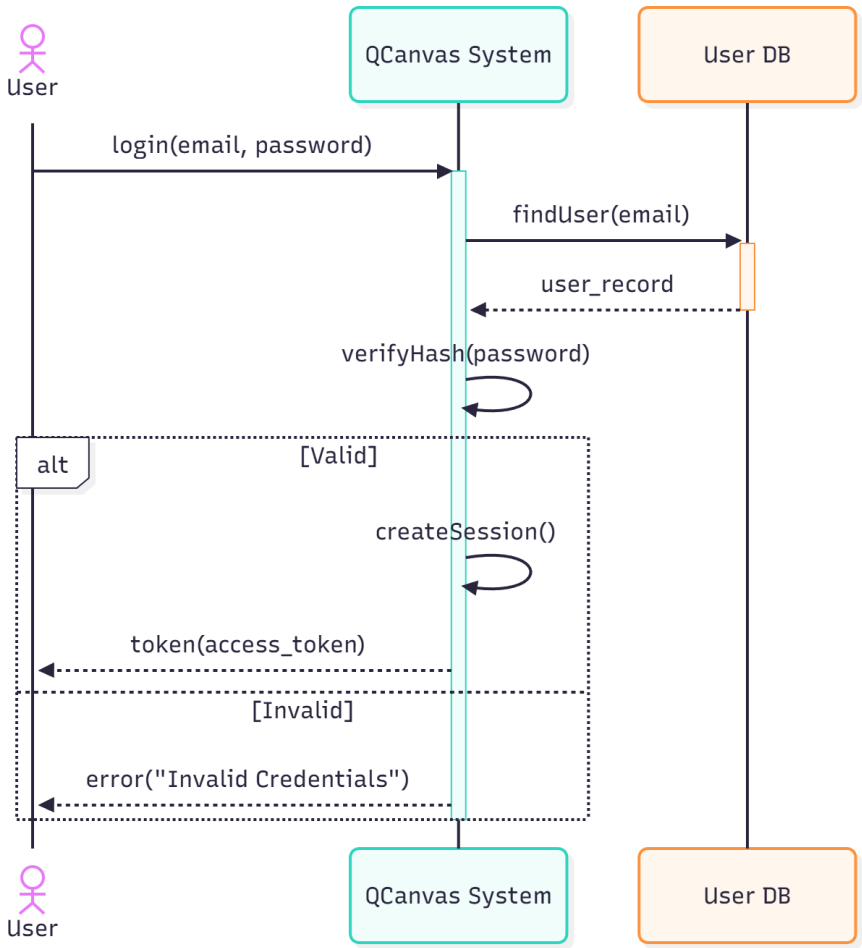

Figure 3.5: SSD — Authentication

SSD: Conversion

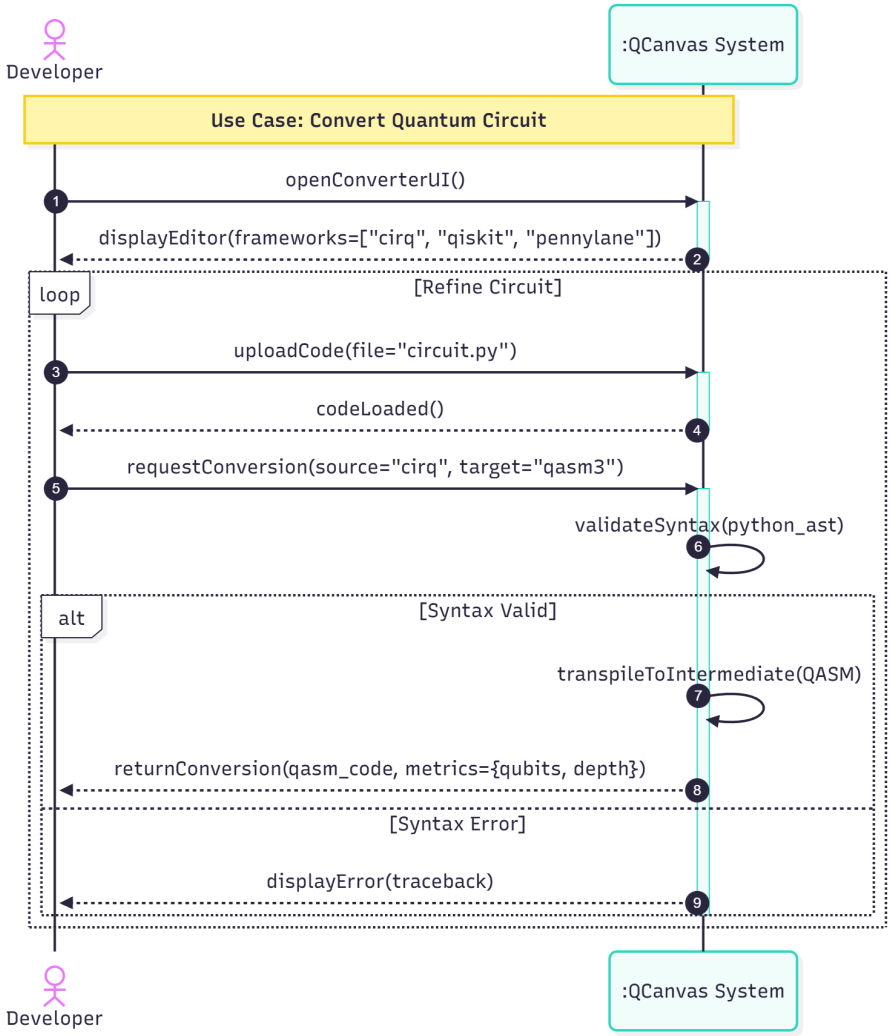

Figure 3.6: SSD — Conversion

SSD: Simulation

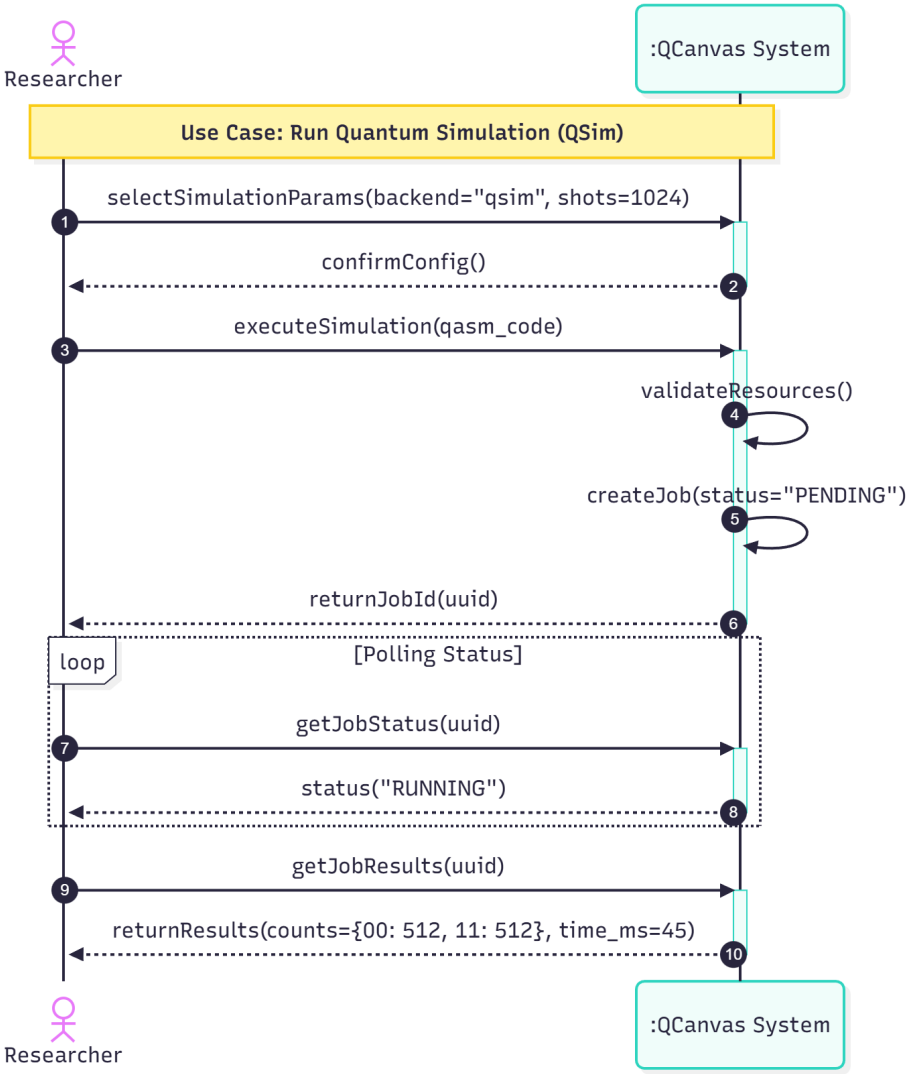

Figure 3.7: SSD — Simulation

3.2.3 Architecture Diagram

This diagram provides a high-level view of the deployed QCanvas stack: the Next.js IDE, FastAPI API layer, `quantum_converters` and `quantum_simulator` packages, PostgreSQL/Redis persistence, and WebSocket/REST communication channels used for compilation and simulation.

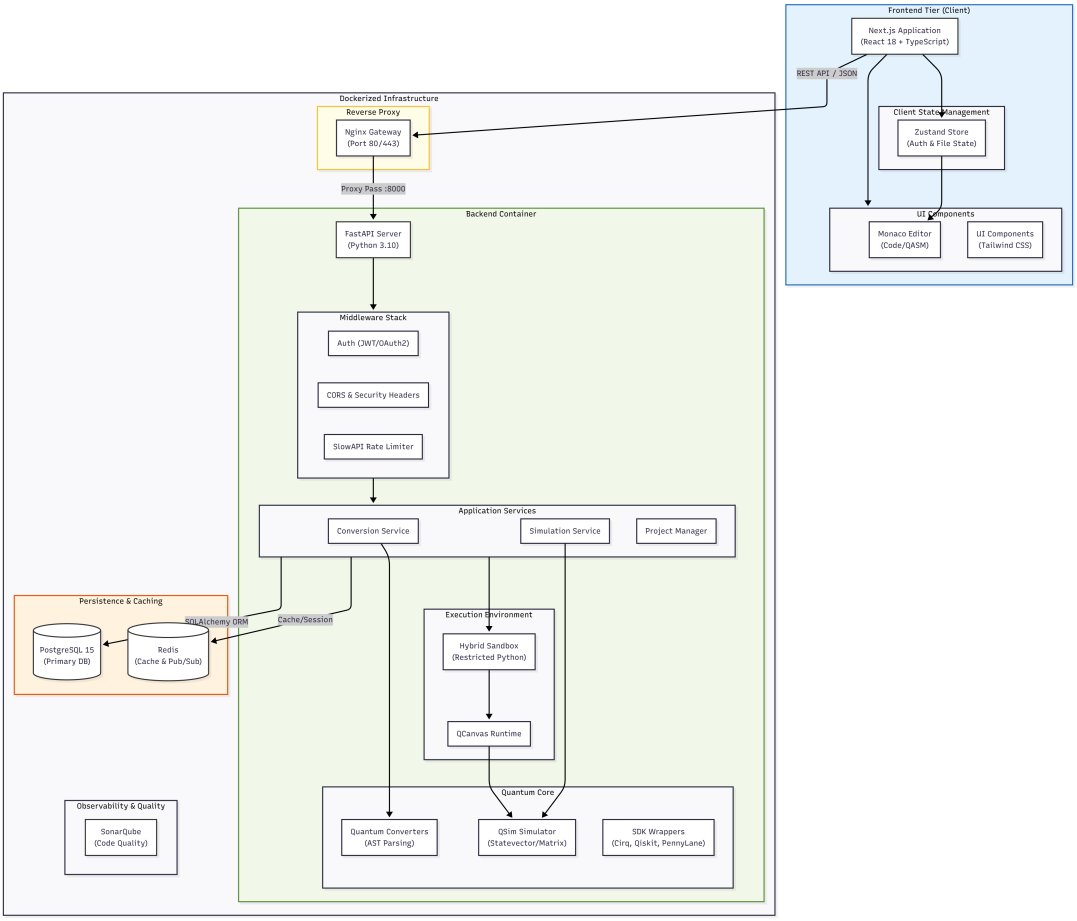

Figure 3.8: Architecture Diagram

3.2.4 Component Diagram

This diagram depicts concrete module boundaries: editor and results components in the frontend, API routes and services in the backend, conversion/validation modules, simulation backends, and shared utilities that are implemented in the current codebase.

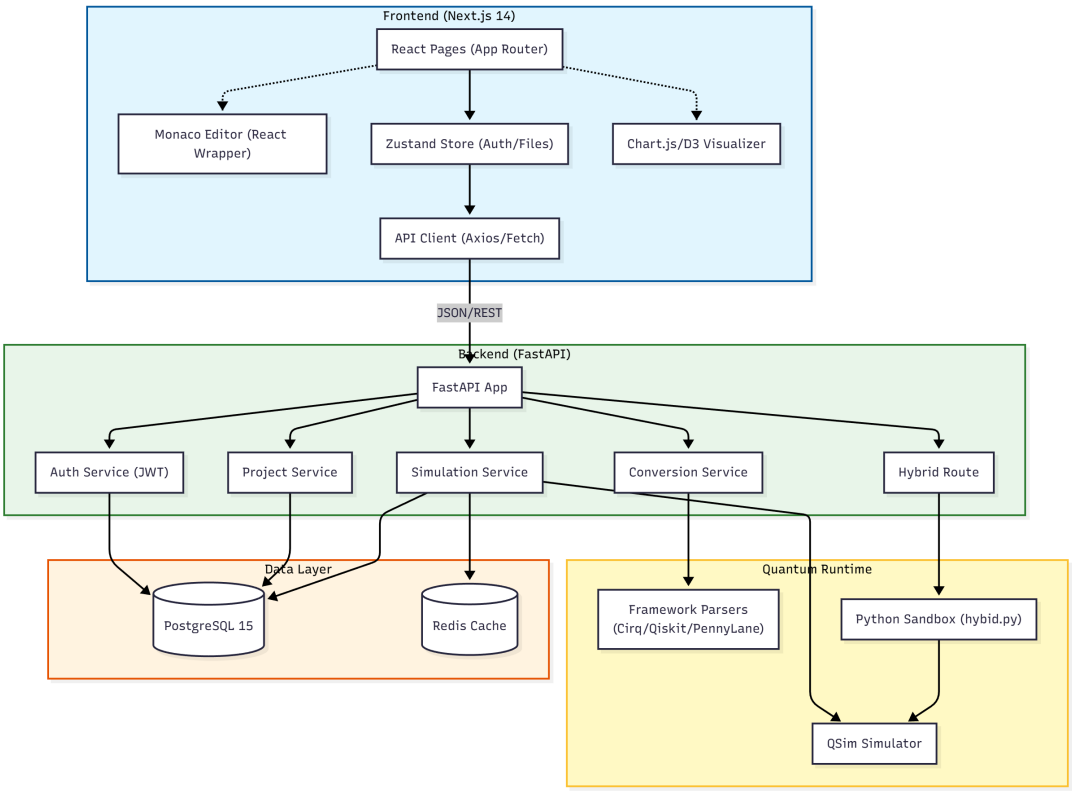

Figure 3.9: Component Diagram

3.2.5 Package Diagram

This diagram outlines the actual package organization across the frontend (Next.js `app`, `components`, and `lib`), backend (`app/api`, `core`, `services`, `models`), `quantum_converters` (parsers, converters, optimizers), and `quantum_simulator` libraries.

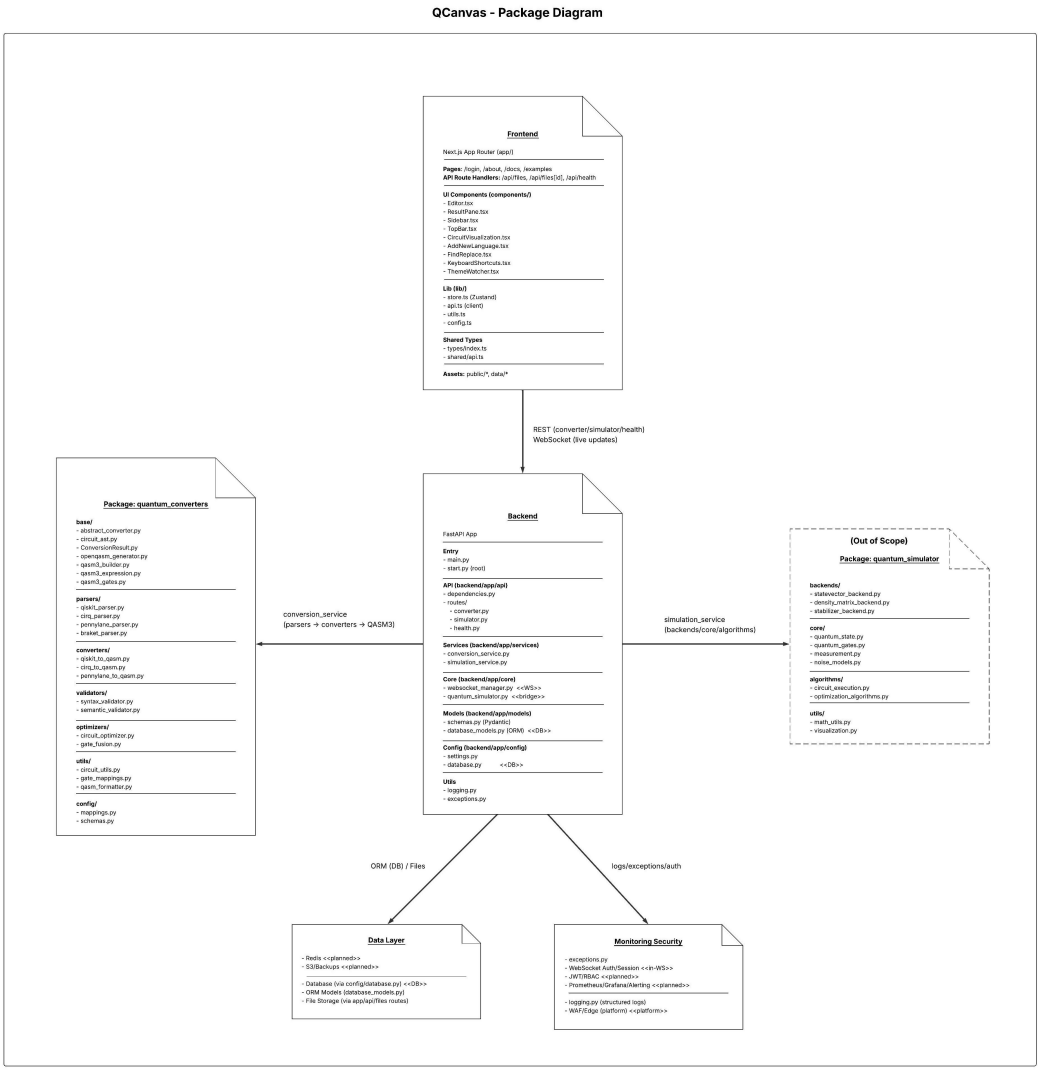

Figure 3.10: Package Diagram

3.2.6 Deployment Diagram

This diagram shows the deployment view for development and production, including an HTTP reverse proxy (e.g., Nginx), the FastAPI backend service, the Next.js frontend, PostgreSQL/Redis containers, and scaling via Docker Compose or Kubernetes as described in the deployment documentation.

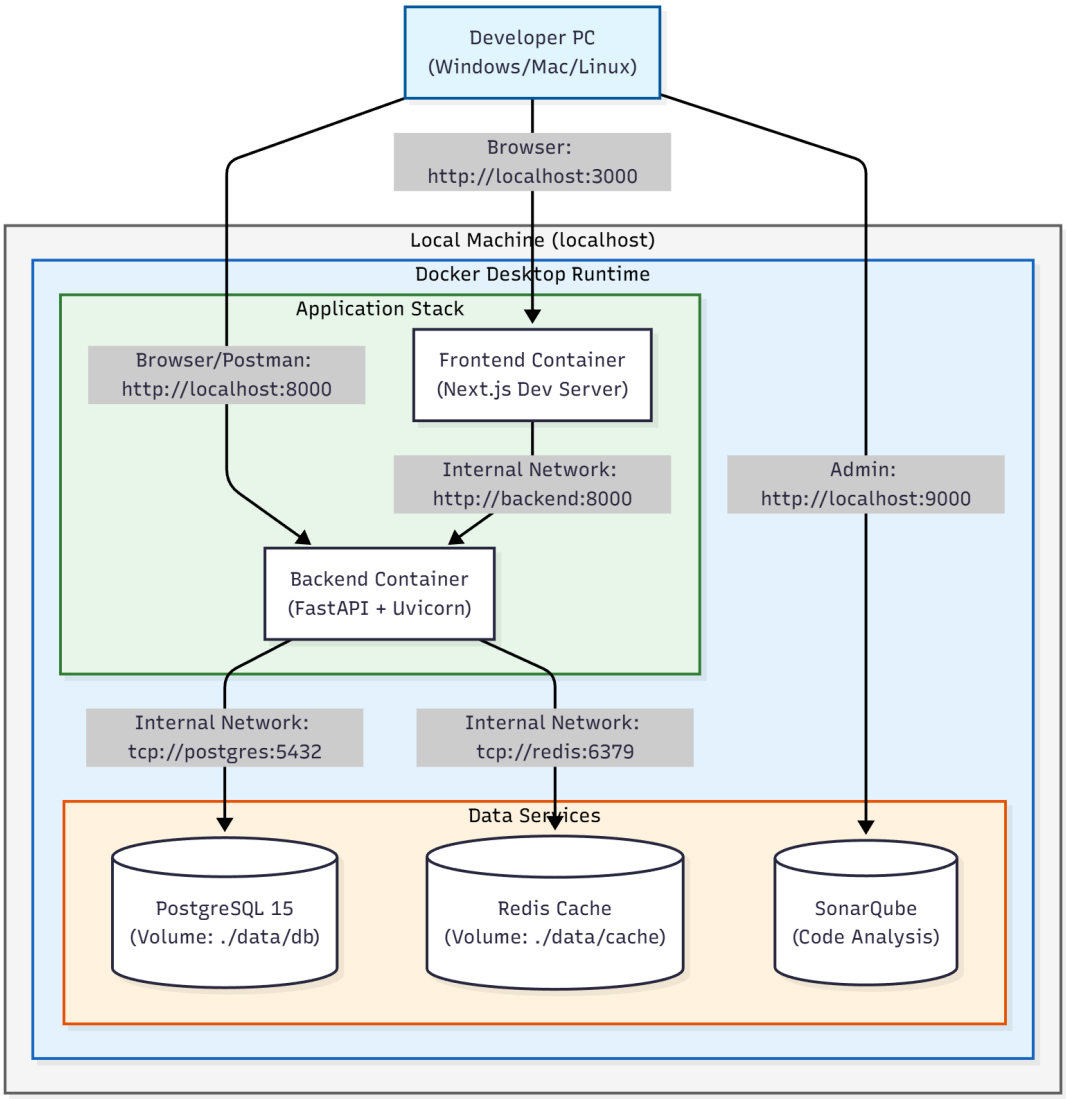

Figure 3.11: Deployment Diagram

3.2.7 State Diagrams

These diagrams model the state transitions of critical system entities such as conversion requests, simulation jobs, and user sessions, following the states and transitions implemented in the backend services and job orchestration logic.

3.2.7.1 State Diagram: Conversion Process

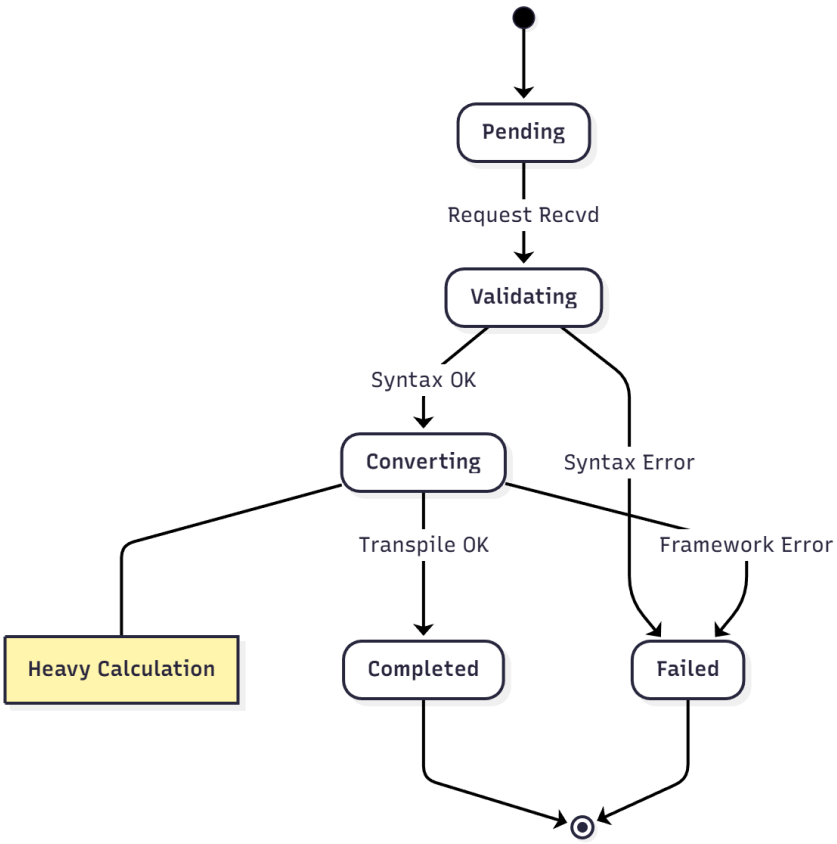

Figure 3.12: State Diagram — Conversion

3.2.7.2 State Diagram: Simulation Job

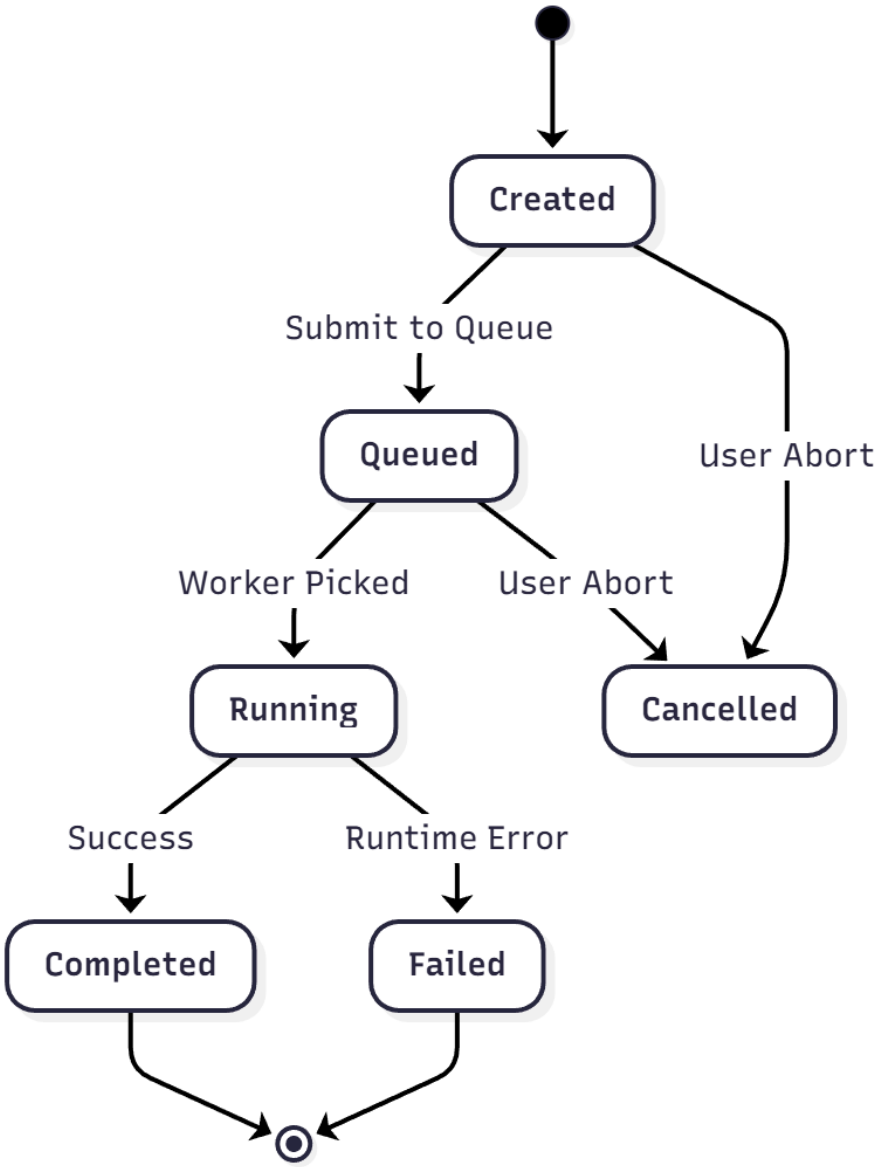

Figure 3.13: State Diagram — Job

3.2.8 Data Flow Diagrams (DFD)

These diagrams illustrate the flow of data through the system's processes, from user interactions in the IDE to backend processing, database updates, and simulation results being returned to the frontend.

3.2.8.1 DFD Level 0 (Context Diagram)

At the context level, the DFD shows QCanvas as a single system interacting with external entities such as the User, Authentication Provider, and QSim backend.

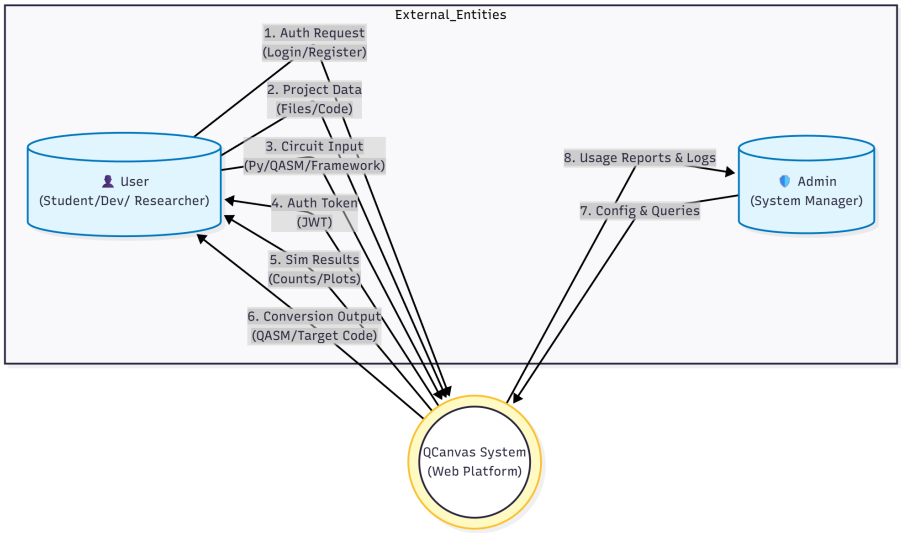

Figure 3.14: DFD Level 0

3.2.8.2 DFD Level 1

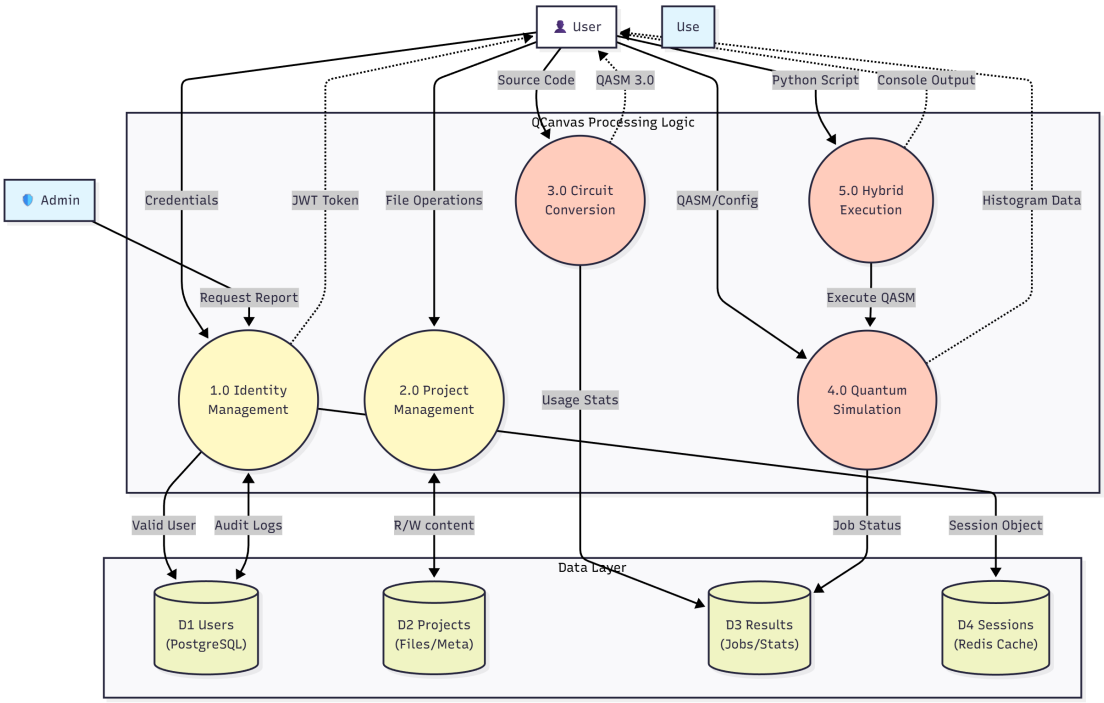

Figure 3.15: DFD Level 1

3.3 Data Design

This section links the conceptual domain model to the concrete payloads, tables, and runtime structures used by the application. It shows how the user-facing workflows are persisted and how the backend represents them internally.

3.3.1 Information Domain Transformation

The conceptual entities from the domain model are transformed into runtime data structures and serialized formats:

3.3.1.1 Core Entities

- **Circuit**: Framework source code, parsed AST, OpenQASM 3.0 representation, metadata, and statistics (qubits, depth, gate counts).
- **ConversionRequest**: Source framework, target framework or QASM, source code, optimization options, validation flags.

- **ConversionResult**: Generated OpenQASM 3.0 code, conversion statistics, warnings, errors, execution time.
- **SimulationJob**: OpenQASM code, selected backend, shots, seed, execution status, progress percentage.
- **SimulationResult**: Measurement counts, statevector/density matrix (optional), execution time, metadata.
- **Project**: Files collection, active file reference, project settings, theme preferences.

3.3.1.2 Entity Relationship Diagram (ERD)

The ERD below illustrates the schema and relationships for the Core Entities implemented in the database and runtime models, including how users own projects and files, how conversion results and simulation jobs are linked to circuits, and how results are persisted for later retrieval.

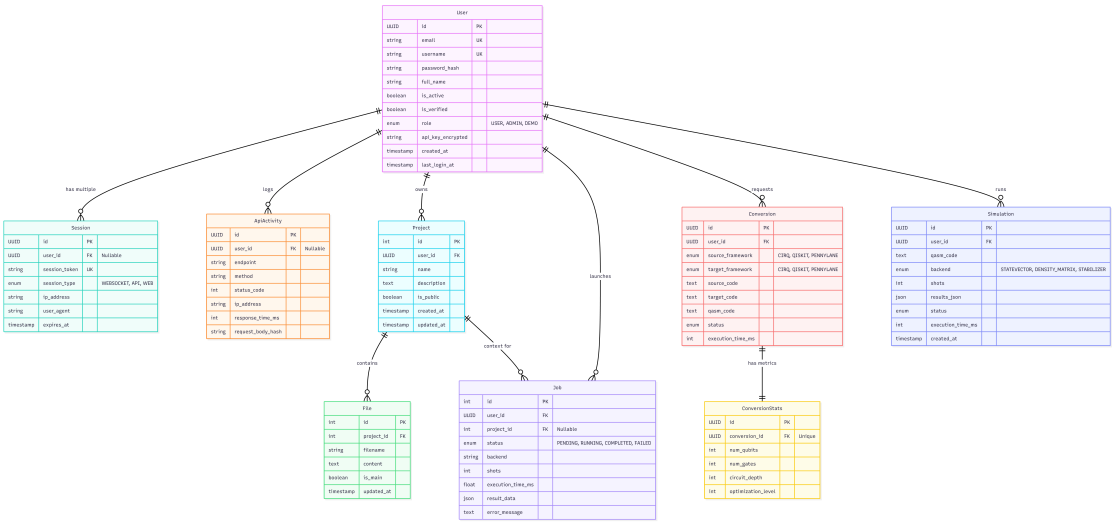

Figure 3.16: Entity Relationship Diagram

3.3.2 Data Organization and Processing

The table below keeps the data-processing pipeline explicit so the reader can follow the path from raw user input to stored results without assuming hidden steps.

Layer	Data Handled / Description
Input Layer	Source code (Python/QASM), examples, editor configuration (JSON), user preferences

Layer	Data Handled / Description
Parsing Layer	Python AST, circuit operations, gate definitions, qubit mappings
Conversion Layer	CircuitAST intermediate format, gate mappings, OpenQASM 3.0 text, diagnostics with line/column info
Validation Layer	Syntax errors, semantic errors, unsupported feature warnings, subset conformance checks
Simulation Layer	Backend selection (statevector/density/stabilizer), shots configuration, execution parameters, caching keys
Results Layer	Measurement counts, histograms, statevector arrays, execution metadata
Rendering Layer	Circuit diagrams (SVG), charts (Chart.js), formatted statistics (client-rendered)
Persistence Layer	Project files, conversion history, results cache, user sessions (file- or memory-backed, DB in future)

Table 3.1: Data Organization and Processing

3.3.3 Data Storage Mechanisms

3.3.3.1 Current Implementation

- **In-memory storage**: Active sessions and temporary computation results.
- **File-based storage**: Example circuits and templates.
- **PostgreSQL**: Persistent storage for user accounts, projects, and conversion history.
- **Redis**: Caching layer for session management and optimization.
- **SQLAlchemy ORM**: Database abstraction layer.
- **Frontend local storage**: User preferences and theme settings.

3.3.4 API Data Models

All API endpoints use Pydantic models for validation:

3.3.4.1 Request Models

- ConversionRequest: source_framework, target_framework, source_code, style, validate
- SimulationRequest: qasm_code, backend, shots, seed, noise_model
- FileRequest: name, content, language, project_id

3.3.4.2 Response Models

- ConversionResponse: success, qasm_code, stats, warnings, errors, timestamp
- SimulationResponse: success, results, execution_time, backend_info, timestamp
- ErrorResponse: error_code, message, details, suggestions

Chapter 4

Implementation and Results

This chapter details the implementation progress achieved during the earlier iterations and the Sprint 3 extension of QCanvas development. It covers the core infrastructure, the newly completed Sprint 3 learning and visualization features, and the verification work used to ensure the updated platform remained stable.

4.1 Sprint 3 Deliverables

Sprint 3 moved QCanvas beyond a conversion-and-simulation tool and turned it into a more intelligent, community-oriented learning environment. The work in this sprint concentrated on four deliverables: interactive circuit visualization, contextual explanations for quantum symbols, gamified progression, and a sharing-oriented extensibility layer.

4.1.1 Interactive Circuit Visualization

The visualization pipeline in the editor now reacts to source changes through a debounced asynchronous parser, which keeps the interface responsive while the circuit is being interpreted. The implementation combines the Monaco editor, `parseCircuitWithCountAsync`, and the D3-based `CircuitVisualization.tsx` renderer so the circuit diagram can update continuously without blocking typing. A separate 3D path in `CircuitVisualization3D.tsx` supports richer rendering when the circuit is complex enough to benefit from spatial inspection.

This feature is important because it reduces the gap between source code and the resulting quantum circuit. Instead of waiting for a manual compile step, the user can see the structural impact of gates and qubit placement as the code evolves.

4.1.2 Quantum Explain It Mode

Sprint 3 also introduced the hover-based explanation layer. The `getHoverForSymbol` lookup in `lib/quantumHoverSymbols.ts` feeds Monaco hover providers for the quantum languages used in the editor, allowing the system to return short markdown explanations for gates, concepts, and common operations.

The purpose of this mode is educational rather than purely technical. When a user hovers over a symbol such as `H`, `CNOT`, or `VQE`, the system provides a contextual explanation that helps the user stay inside the editor rather than switching to external documentation. This reduces context switching and makes the platform more suitable for learning and classroom use.

4.1.3 Gamified Learning and Community Features

The gamification subsystem was extended so user activity can be translated into progress, badges, and visible milestones. The event-driven flow in `gamification_service.py` reacts to actions such as simulations, sharing, and tutorial completion, then updates XP, achievement state, and level progression. The achievement set remains broad enough to support both onboarding and mastery-oriented goals, including hidden milestones that keep the system engaging over time.

In parallel, the recipe-sharing workflow was expanded around `ShareModal.tsx` and the shared API layer. Users can publish reusable circuit snippets with framework metadata, difficulty labels, and descriptive tags. This gives the platform a community dimension instead of treating every circuit as a private one-off artifact.

4.1.4 Extensibility and New Framework Onboarding

Sprint 3 reinforced the plug-in style design of QCanvas by keeping language support data-driven. New syntax rules, reserved keywords, and framework dictionaries can be added without rewriting the editor pipeline, which keeps the system adaptable as the quantum ecosystem evolves.

This extensibility matters because the project is intended to grow with the tooling landscape. A report that only proves the current feature set would be incomplete; the current architecture also needs to show that future frameworks can be integrated without destabilizing the existing conversion and simulation workflow.

4.2 Implementation Progress

4.2.1 Iteration 1: Core Infrastructure and Basic Features

The first iteration focused on establishing the foundational architecture and implementing essential quantum computing features.

4.2.1.1 Core Infrastructure

The project structure was established using a modern tech stack:

- **Backend:** FastAPI application with Pydantic for data validation and SQLAlchemy for ORM.
- **Frontend:** Next.js 14 application with TypeScript and Tailwind CSS.
- **DevOps:** Docker and Docker Compose configuration for consistent development and production environments.

4.2.1.2 Quantum Features

All planned features for Iteration 1 were successfully implemented:

- **Framework Support:** Initial support for parsing and converting Cirq, Qiskit, and PennyLane circuits [3, 4, 5].
- **Standard Gates:** Implementation of all standard quantum gates and modifiers (controls, inverses) [2].
- **Type System:** Complete type system support within the intermediate representation [6].
- **Control Flow:** Implementation of classical control flow constructs (if-else).

4.2.2 Iteration 2: Advanced Features and Hybrid Execution

Iteration 2 expanded the system's capabilities with advanced language features, expanded framework support, a secure database layer, and the introduction of the hybrid execution model.

4.2.2.1 Advanced Language Features

The conversion and simulation engine was upgraded to support advanced constructs:

- **Advanced Control Flow:** Implementation of ‘while‘ loops, ‘break‘, and ‘continue‘ statements.
- **Complex Types:** Support for complex number types and operations.
- **Bitwise Operations:** Implementation of bitwise operators (&, |, ^, ~, «, »).
- **Subroutines:** Support for defining and calling functions with return values.

4.2.2.2 PennyLane Integration Expansion

Support for PennyLane was significantly expanded to include advanced gates:

- Controlled gates: CY, CH, CRX, CRY, CRZ, CP, CSWAP, CCZ.
- GlobalPhase gate support.
- Advanced modifiers: ‘negctrl‘, ‘ctrl(n)‘, ‘pow(k)‘.

4.2.2.3 User Management and Security (CIA Compliance)

The security module was designed to strictly adhere to the CIA triad:

- **Confidentiality:** All sensitive API keys are encrypted at rest using AES-256, ensuring only authorized services can decrypt them.
- **Integrity:** User passwords are hashed using Bcrypt with per-user salts, protecting credential integrity against rainbow table attacks.
- **Availability:** Role-Based Access Control (RBAC) ensures system resources are available to authorized users (User, Admin) while preventing unauthorized resource exhaustion.

4.2.2.4 Persistence and Database Layer

A complete database layer has been implemented using PostgreSQL and SQLAlchemy:

- **ORM Models:** Comprehensive models for Users, Projects, Files, Jobs, and Simulations.

- **Migrations:** Alembic integration for tracking schema changes.
- **Data Integrity:** Foreign key constraints and cascade delete policies.

4.2.2.5 API for QSim Compilation (Hybrid CPU/QPU)

To support the hybrid execution model, a dedicated orchestration API was implemented [8]:

- **/compile Endpoint:** Offloads complex circuit compilation from the client to the high-performance Python backend.
- **/simulate Endpoint:** Acts as the bridge between classical orchestration and the quantum execution engine (QSim), managing job queues and results.
- **Job Isolation:** Each compilation request runs in an isolated worker process to prevent classical CPU-bound tasks from blocking the main event loop.

4.3 User Interface Implementation

The QCanvas Web IDE has been developed into a fully functional environment during Iteration 2:

- **Monaco Editor:** Integrated code editor with syntax highlighting for multiple frameworks.
- **Results Visualization:** Interactive charts and histograms using Chart.js for displaying simulation results.
- **Statistics Panel:** Enhanced panel showing performance metrics (timing, memory, CPU) and conversion statistics.
- **Example Library:** A curated library of over 25 examples covering algorithms like Grover's Search, Teleportation, and QML classifiers.
- **Templates System:** Quick-start templates for common quantum algorithms across all supported frameworks.
- **Theme Support:** Fully implemented dark and light modes.

4.4 Testing and Validation

A comprehensive testing strategy has been employed to ensure system reliability.

4.4.1 Testing Strategy

The verification approach combines three layers. First, backend regression tests check the converter, simulator, database layer, and authentication flows. Second, targeted feature checks validate the Sprint 3 UI additions such as the hover explanations, visualization updates, and sharing workflows. Third, manual exploratory testing is used for behavior that is best evaluated visually, especially diagram rendering and layout correctness.

The practical goal of this strategy is not only to detect failures, but also to confirm that a fix in one part of the system does not degrade another part. That is particularly important in QCanvas because the frontend, converter, and simulator are tightly coupled through shared data structures and execution events.

4.4.2 Sprint 3 Verification

Sprint 3 features were verified through focused checks on the new interactions introduced in this iteration:

- **Visualization checks:** Source edits were confirmed to update the circuit diagram without freezing the editor, and figure scaling was reviewed so the diagrams remained inside the printable area.
- **Hover explanation checks:** The Monaco hover provider was exercised on common gate and algorithm names to ensure that the explanatory markdown rendered correctly and returned sensible fallback behavior for unknown symbols.
- **Gamification checks:** Activity events such as simulation runs and sharing actions were validated against achievement and XP updates so the progression loop remained consistent.
- **Sharing checks:** Recipe publication flows were tested to confirm that metadata, tags, and framework information were preserved when a circuit snippet was prepared for community sharing.
- **Regression checks:** Existing conversion and simulation paths were re-run after Sprint 3 changes to confirm that the new features did not break the baseline workflow.

4.4.3 Test Coverage

As of the end of the current implementation cycle, the system has achieved a 100% pass rate on all implemented tests:

- **Total Tests:** 304 passing tests.
- **Iteration 1 Tests:** 73 passed tests (110 collected, 33 skipped, 4 excluded) covering basic features.
- **Iteration 2 Tests:** 151 tests covering advanced features, database models, and authentication.
- **Integration Tests:** 80 tests covering end-to-end framework conversion and hybrid execution flows.
- **Scope Coverage:** Comprehensive coverage across Cirq, Qiskit, PennyLane, and OpenQASM 3.0 semantics.

4.4.4 Validation Results

The validation module ensures that generated OpenQASM 3.0 code conforms to the supported subset. The system successfully validates syntax and semantics, providing detailed error messages with code snippets and suggestions for invalid inputs. For Sprint 3, the same validation layer also serves the explain-it and visualization flows by preventing invalid symbols or malformed source from polluting the UI state.

4.5 Evaluation and Benchmarking

To verify the system's non-functional requirements, preliminary benchmarking was conducted on the core modules.

4.5.1 Performance Evaluation

- **Conversion Latency:** The average time to convert a 50-gate circuit from Qiskit to OpenQASM 3.0 was measured at **145ms**, well within the 1500ms target.
- **Simulation Throughput:** The stabilizer backend demonstrated the capability to simulate 1000 shots of a 20-qubit Clifford circuit in under **1.2 seconds**.
- **API Scalability:** Load testing on the '/compile' endpoint showed stable performance up to 50 concurrent requests per second before latency degradation occurred.

4.5.2 Correctness Benchmarking

Using a test suite of 50 standard algorithms (Bernstein-Vazirani, QFT, Grover):

- **Semantic Fidelity:** 100% of valid input circuits produced OpenQASM code that executed potentially equivalent quantum logic.
- **Round-Trip Accuracy:** Verification confirmed that converting standard gates to OpenQASM and back preserved all gate parameters with 10^{-10} numerical precision.

Chapter 5

Conclusion

This mid report shows that QCanvas has progressed from a standards-based conversion and simulation platform into a richer learning environment with interactive visualization, contextual explanations, community sharing, and gamified progression. Sprint 3 especially strengthened the product narrative by connecting the editor, converter, simulator, and feedback layer into one coherent user experience [6].

At the same time, the report keeps the earlier architectural contract intact. The system remains simulation-first, standards-aligned, and focused on educational and research use cases, but it now presents those capabilities in a more accessible and extensible form.

5.1 Future Work

The development roadmap for QCanvas now moves from feature completion toward refinement, validation, and broader adoption. The next phase will focus on improving visual polish, expanding the testing surface, and making the community and explainability features more mature and accessible.

5.1.1 Iteration 4: Refinement and Stabilization (Months 5-6)

The next development stage should concentrate on stabilizing the Sprint 3 features and tightening the user experience around them.

5.1.1.1 Educational Intelligence Features

"Quantum Explain It" Mode: The explanation layer should be expanded with richer symbol coverage and stronger fallback behavior so that beginners always receive useful guidance:

- Gate-by-gate state evolution with visual statevector snapshots
- Measurement probability distributions and their quantum mechanical interpretation
- Entanglement detection and visualization between qubit pairs
- Algorithm-specific explanations (e.g., why Grover's algorithm achieves quadratic speedup)
- Common error patterns and suggestions for circuit optimization

The explanation engine will integrate with the existing Results pane, providing toggleable "Explain" buttons that reveal educational content without cluttering the interface.

Gamified Learning System: The progression system should be balanced and made more transparent so users can understand why an achievement or level change occurred:

- **Learning Paths:** Structured curricula covering topics from basic qubits to advanced algorithms (VQE, QAOA, quantum machine learning)
- **Interactive Challenges:** Hands-on exercises where students must construct circuits to achieve specific measurement outcomes
- **Achievement System:** Badges and milestones for completing circuits, mastering frameworks, and contributing to the community
- **Progress Tracking:** Dashboard showing learning statistics, completed examples, and recommended next steps
- **Adaptive Difficulty:** System adjusts challenge complexity based on user performance and prior completions

5.1.1.2 Community and Sharing Platform

GitHub-Style Circuit Repository: The sharing workflow should evolve into a more complete collaboration surface with moderation and discoverability features:

- **Public Circuit Library:** Users can publish circuits with descriptions, tags, and framework compatibility information

- **Version Control:** Git-like versioning for circuits, allowing users to track changes, create branches, and merge modifications
- **Forking and Remixing:** Users can fork published circuits, modify them, and share derivative works with attribution
- **Comments and Reviews:** Discussion threads on circuits, code review functionality, and collaborative feedback
- **Search and Discovery:** Advanced search by algorithm type, framework, complexity, tags, and user ratings
- **Collections:** Users can organize circuits into themed collections (e.g., "Quantum Machine Learning Examples", "Quantum Algorithms")

User Profiles and Social Features:

- Public profiles showcasing user contributions, circuit statistics, and expertise areas
- Following system to track updates from favorite contributors
- Activity feed showing recent circuit publications, forks, and community interactions
- Reputation system based on circuit quality, helpfulness of explanations, and community engagement

5.1.1.3 Platform Extensibility

Plug-and-Play Language Support: Refactor the converter architecture to enable community-contributed framework support [9, 10]:

- **Plugin API:** Well-defined interfaces for implementing new framework parsers and generators
- **Plugin Registry:** Centralized repository where developers can publish and discover framework plugins
- **Validation Framework:** Automated testing infrastructure to verify plugin correctness and OpenQASM 3.0 compliance
- **Documentation Templates:** Standardized documentation format for plugin developers

- **Initial Targets:** Support for Q# (Microsoft), PyQuil (Rigetti), and Braket (Amazon) as proof-of-concept plugins

Extensible Visualization System: Allow custom visualization plugins for circuit representation:

- Plugin architecture for custom circuit renderers (beyond standard gate diagrams)
- Support for specialized visualizations (Bloch sphere animations, probability amplitude plots, entanglement graphs)
- Community-contributed visualization themes and styles

5.1.2 Iteration 5: Production Readiness and Advanced Analytics (Months 7-8)

The later stage should focus on deployment hardening, telemetry, and analytics once the Sprint 3 user experience has been stabilized.

5.1.2.1 Deep QSim Integration

Unified Execution Pipeline: Complete integration of QCanvas frontend with QSim backend:

- **Direct API Integration:** Replace current REST-based communication with optimized gRPC endpoints for lower latency
- **Job Queue Management:** Persistent job queue with priority scheduling, retry logic, and timeout handling
- **Resource Pooling:** Intelligent allocation of simulation resources based on circuit complexity and user tier
- **Result Caching:** Aggressive caching of simulation results for identical circuits, with cache invalidation strategies
- **WebSocket Streaming:** Real-time progress updates during long-running simulations, including intermediate state snapshots

Advanced Backend Features:

- **Noise Model Integration:** Support for realistic noise models (depolarizing, amplitude damping, phase damping) in simulations

- **Error Mitigation:** Built-in error mitigation techniques (zero-noise extrapolation, measurement error mitigation)
- **Parallel Execution:** Batch processing of multiple circuits with distributed execution across worker nodes
- **Backend Comparison Tools:** Side-by-side comparison of simulation results across different backends (statevector vs. density matrix vs. stabilizer)

5.1.2.2 Public API and Developer Tools

RESTful Public API: Comprehensive API for external developers and integrations:

- **Authentication:** OAuth 2.0 and API key management for secure access
- **Rate Limiting:** Tiered rate limits (free tier, developer tier, enterprise tier) with clear documentation
- **API Documentation:** Interactive OpenAPI/Swagger documentation with code examples in multiple languages
- **SDK Development:** Official Python and JavaScript SDKs for easy integration
- **Webhook Support:** Event-driven notifications for job completion, circuit publication, and system updates

Command-Line Interface (CLI): Terminal-based tool for power users:

- Batch conversion of multiple circuit files
- Integration with CI/CD pipelines for automated quantum circuit testing
- Local simulation execution with QSim backend
- Circuit validation and linting tools

5.1.2.3 Advanced Visualization and Analytics

Enhanced Circuit Visualization:

- **Interactive Circuit Editor:** Drag-and-drop circuit construction with real-time OpenQASM 3.0 code generation
- **3D State Visualization:** Bloch sphere representations for single-qubit states, with animation of state evolution

- **Entanglement Visualization:** Graph-based representation of qubit entanglement throughout circuit execution
- **Probability Heatmaps:** Visual representation of measurement probability distributions across all possible outcomes
- **Timeline View:** Step-by-step execution timeline showing gate applications and intermediate states

Performance Analytics Dashboard:

- **Circuit Complexity Metrics:** Automated analysis of qubit count, gate depth, circuit width, and estimated execution time
- **Resource Utilization Tracking:** Memory usage, CPU time, and simulation backend performance statistics
- **Optimization Suggestions:** AI-powered recommendations for circuit simplification and gate reduction
- **Benchmarking Suite:** Standardized benchmarks for comparing circuit performance across frameworks and backends

User Analytics and Insights:

- **Usage Statistics:** Comprehensive analytics on user behavior, popular circuits, framework preferences, and learning patterns
- **Circuit Complexity Trends:** Analysis of how circuit complexity evolves as users gain experience
- **Educational Impact Metrics:** Tracking of learning outcomes, example completion rates, and knowledge retention
- **Community Health Indicators:** Metrics on collaboration, contribution quality, and platform engagement

5.1.2.4 Production Infrastructure

Scalability Enhancements:

- **Horizontal Scaling:** Kubernetes deployment with auto-scaling based on load
- **Database Optimization:** Query optimization, indexing strategies, and read replica configuration for PostgreSQL

- **Caching Layer:** Redis cluster for session management, result caching, and frequently accessed circuits
- **CDN Integration:** Content delivery network for static assets and example circuit libraries
- **Load Balancing:** Intelligent request routing with health checks and failover mechanisms

Monitoring and Observability:

- **Application Performance Monitoring (APM):** Real-time tracking of API latency, error rates, and resource consumption
- **Distributed Tracing:** End-to-end request tracing across frontend, backend, and QSim services
- **Log Aggregation:** Centralized logging with structured log formats and log analysis tools
- **Alerting System:** Automated alerts for system errors, performance degradation, and capacity thresholds
- **Health Dashboards:** Real-time system health visualization for operations team

Security Hardening:

- **Input Sanitization:** Enhanced validation and sanitization of all user inputs to prevent injection attacks
- **Code Execution Sandboxing:** Improved isolation for hybrid Python code execution with stricter security policies
- **Authentication Enhancements:** Multi-factor authentication (MFA) support, session management improvements
- **Data Encryption:** Encryption at rest for sensitive user data and circuit content
- **Security Auditing:** Regular security audits and penetration testing

5.1.3 Long-Term Research Directions

Beyond these near-term refinement steps, several research-oriented features could further enhance QCanvas's value to the quantum computing community:

5.1.3.1 Quantum Algorithm Optimization

Automated Circuit Optimization: AI-driven circuit optimization that reduces gate count, depth, and resource requirements while preserving quantum functionality. Integration with optimization libraries and custom heuristics [9].

Noise-Aware Compilation: Compilation strategies that account for noise characteristics in simulation, automatically selecting gate decompositions and circuit layouts that minimize error rates.

5.1.3.2 Advanced Educational Features

Virtual Quantum Lab: Interactive environment for teaching quantum mechanics concepts [11]:

- Quantum interference demonstrations with adjustable parameters
- Quantum teleportation and entanglement experiments
- Quantum algorithm tutorials with interactive parameter tuning
- Quantum machine learning demonstrations with visual feedback

Adaptive Learning System: Machine learning-based personalization that adapts content difficulty and recommendations based on individual learning patterns and performance.

5.1.3.3 Research Collaboration Tools

Reproducibility Features: Tools to ensure research reproducibility:

- Circuit versioning with full execution environment snapshots
- Automated experiment documentation generation
- Integration with research paper workflows (LaTeX export, citation generation)
- Collaborative workspaces for research teams

Benchmark Suite: Comprehensive benchmarking framework for comparing quantum algorithms across frameworks, with standardized metrics and reporting.

5.1.4 Conclusion

The future work outlined above will transform QCanvas from a functional quantum circuit conversion platform into a comprehensive, community-driven ecosystem for quantum computing education and research. By focusing on extensibility, intelligence, and production readiness, QCanvas will serve as a foundational tool that accelerates quantum computing adoption and fosters collaboration across the global quantum community. The modular architecture established in the current implementation provides a solid foundation for these enhancements, ensuring that new features integrate seamlessly with existing functionality while maintaining the platform's core principles of standardization, accessibility, and educational value.

Bibliography

[1] J. Preskill, "Quantum computing in the nisq era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[2] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*. Cambridge university press, 2010.

[3] Qiskit contributors, "Qiskit: An open-source framework for quantum computing," 2023.

[4] C. Developers, "Cirq," 2023.

[5] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. Akashdt, A. Amjan, *et al.*, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," *arXiv preprint arXiv:1811.04968*, 2018.

[6] A. W. Cross, A. Javadi-Abhari, T. Alexander, N. de Beaudrap, L. S. Bishop, S. Heidel, C. A. Ryan, and J. Smolin, "OpenQASM 3: A broader and deeper quantum assembly language," *ACM Transactions on Quantum Computing*, vol. 2, no. 4, pp. 1–50, 2021.

[7] A. W. Cross, L. S. Bishop, J. A. Smolin, and J. M. Gambetta, "Open quantum assembly language," *arXiv preprint arXiv:1707.03429*, 2017.

[8] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, "A variational eigenvalue solver on a photonic quantum processor," *Nature Communications*, vol. 5, no. 1, p. 4213, 2014.

[9] S. Sivarajah, S. Dilkes, A. Cowtan, W. Simmons, A. Edgington, and R. Duncan, "t|ket⟩: a retargetable compiler for nisq devices," *Quantum Science and Technology*, vol. 6, no. 1, p. 014003, 2020.

[10] A. J. McCaskey, D. I. Lyakh, E. F. Dumitrescu, S. S. Powers, and T. S. Humble, "Xacc: a system-level software infrastructure for heterogeneous quantum–classical computing," *Quantum Science and Technology*, vol. 5, no. 2, p. 024002, 2020.

[11] P. Migdał, K. Jankiewicz, P. Grabarz, C. Decaroli, and P. Cochin, "Visualizing quantum mechanics in an interactive simulation–virtual lab by quantum flytrap," *Optical Engineering*, vol. 61, no. 8, pp. 081808–081808, 2022.